Model Theory

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Overview . . . 2
Pages in this section . . . 2
How to read these pages . . . 3
Overview . . . 3
Process equations . . . 3
Switches and parameters . . . 4
Implementation . . . 4
Related pages . . . 4
Overview . . . 5
Process equations . . . 5
Switches and parameters . . . 6
Implementation . . . 6
Related pages . . . 6
Overview . . . 7
Process equations . . . 7
Switches and parameters . . . 8
Implementation . . . 9
Related pages . . . 9
Overview . . . 10
Process equations . . . 10
Switches and parameters . . . 11
Implementation . . . 12
Related pages . . . 12
Overview . . . 12
Process equations . . . 12
Switches and parameters . . . 14
Implementation . . . 14
Related pages . . . 15
Overview . . . 15
Process equations . . . 15
Land use management . . . 16
Switches and parameters . . . 17
Implementation . . . 18
Related pages . . . 18
Overview . . . 18
Process equations . . . 18
In-stream nutrient routing (QUAL2E) . . . 21
Switches and parameters . . . 21
Implementation . . . 22
Related pages . . . 22
Overview . . . 22
Process equations . . . 22
Switches and parameters . . . 24
Implementation . . . 25

Related pages . . . 25
Overview . . . 25
Process equations . . . 25
Switches and parameters . . . 27
Implementation . . . 27
Related pages . . . 28
Overview . . . 28
Process equations . . . 28
Switches and parameters . . . 29
Implementation . . . 30
Related pages . . . 30
Overview . . . 31
Process equations . . . 31
Switches and parameters . . . 32
Implementation . . . 33
Related pages . . . 34
Overview . . . 34
Process equations . . . 34
Switches and parameters . . . 35
Implementation . . . 36
Related pages . . . 36
Overview . . . 37
Process equations . . . 37
Switches and parameters . . . 38
Implementation . . . 39
Related pages . . . 39
Overview . . . 40
Process equations . . . 40
Switches and parameters . . . 41
Implementation . . . 41
Related pages . . . 42
Overview . . . 42
Process equations . . . 42
Switches and parameters . . . 44
Implementation . . . 45
Related pages . . . 45

Overview

This section describes how SWAT+ represents the physical and management processes that move water, sediment, nutrients, and other constituents through a watershed. Each page focuses on one process cluster, lists the governing relationships, the basin codes and parameters that switch behaviour, and the source modules in `swatplus/src/` that implement it.

The clusters group together because they share state, run in the same daily order, or are configured through the same input files. The water balance cluster (HRU hydrology, soils, climate, erosion) feeds the routing cluster (channels, aquifers, reservoirs and wetlands, recall objects). The biogeochemistry cluster (plant growth, nutrient cycling, carbon, pesticides, salts and constituents) sits inside the HRU. Decision tables drive management and release rules across all clusters.

Pages in this section

Cluster	Page	What it covers
Water balance	Overview and water balance	Daily HRU water balance, object flow chart, sequencing

Cluster	Page	What it covers
Water balance	Climate and weather	Measured, simulated, and lapsed weather; PET methods
Water balance	Hydrology	Runoff (CN, Green-Ampt), lateral flow, percolation, channel routing
Water balance	Soil water and temperature	Soil profile water, soil temperature, frozen soil
Water balance	Erosion and sediment	MUSLE on land, channel erosion and deposition
Routing	Channels	Channel routing methods, sediment transport, in-stream water quality
Routing	Aquifers	1D recession aquifer and 2D `gwflow` cell-based groundwater
Routing	Reservoirs and wetlands	Reservoir water balance, release rules, sediment and nutrient processes; HRU wetlands
Routing	Recall and point sources	Recall time series, export coefficients, delivery ratios
Routing	Decision tables	Runtime condition evaluation and action firing
Biogeochemistry	Plant growth and land use management	Heat units, biomass, yield, scheduled management
Biogeochemistry	Nutrient cycling	Soil N and P pools, transformations, transport
Biogeochemistry	Carbon	Static, CFARM, and Century carbon models
Biogeochemistry	Pesticides	Application, partitioning, decay, transport
Biogeochemistry	Salts and constituents	Salt ion chemistry and generic constituents

How to read these pages

Each page follows the same shape. The Overview names the process and the role it plays in the simulation. Process equations give the governing relationships in plain math. Switches and parameters lists the `codes.bsn` and `parameters.bsn` entries that change the result, with defaults from the source. Implementation lists the source files in `swatplus/src/` that perform the calculation. Related pages cross-link to the input and output references.

The Fortran source at `swatplus/src/` is the source of truth. Where this section conflicts with the source, the source wins. Anything still uncertain is marked `TODO: verify`.

Overview

SWAT+ solves a daily water balance for every hydrologic response unit (HRU). The land-phase balance partitions precipitation into surface runoff, lateral flow, percolation, evapotranspiration, and change in soil water storage. Channel routing, aquifer recharge, and reservoir storage build on top of this HRU-level balance. The HRU loop in `hru_control.f90` is the entry point for the daily land-phase calculation.

Process equations

The daily water balance for an HRU is

```
P = Q_surf + Q_lat + Q_perc + ET + dS
```

Term	Meaning
`P`	Daily precipitation reaching the soil surface (after canopy interception and snow processes)
`Q_surf`	Surface runoff routed to the channel during the day
`Q_lat`	Lateral subsurface flow leaving the soil profile
`Q_perc`	Percolation below the bottom of the soil profile (recharge to the shallow aquifer)
`ET`	Actual evapotranspiration (canopy evaporation, transpiration, soil evaporation, sublimation)
`dS`	Change in soil profile water storage for the day

Subsidiary terms that the model also tracks at the HRU level and accounts for in the balance:

- Tile flow `Q_tile`, computed in the percolation pass.
- Wetland and pothole storage changes (`wet_control`, ponded HRU types).
- Snow water equivalent, accumulated and melted in `sq_snom.f90`.
- Surface runoff stored over one day through the `surlag` lag (`sq_surfst.f90`).

The HRU water balance feeds the basin-scale and channel-scale balances. Outputs at HRU, landscape unit (LSU), and basin levels are written by the `*_output.f90` modules.

Switches and parameters

Switch / parameter	File	Effect
`bsn_cc%pet`	`codes.bsn`	Selects PET method (see Climate and Weather)
`bsn_cc%gampt`	`codes.bsn`	Curve number vs Green-Ampt infiltration (see Hydrology)
`bsn_cc%rte`	`codes.bsn`	Channel routing method
`bsn_prm%surlag`	`parameters.bsn`	Surface runoff lag, default 4.0 days
`bsn_prm%ffcb`	`parameters.bsn`	Initial soil water as fraction of field capacity, default 0.0
`hyd_db%esco`, `hyd_db%epco`	`hydrology.hyd`	Soil evap and plant uptake compensation factors
`hyd_db%canmx`	`hydrology.hyd`	Maximum canopy storage (mm)
`hyd_db%lat_ttime`	`hydrology.hyd`	Lateral flow travel time

Implementation

Source modules in `swatplus/src/` that implement the land-phase water balance:

- `hru_control.f90` orchestrates the daily HRU loop: canopy interception, snow, PET, surface runoff, percolation, lateral storage, evapotranspiration, and output accumulation.
- `sq_canopyint.f90` computes canopy interception storage.
- `sq_snom.f90` accumulates and melts snowpack.
- `sq_volq.f90` dispatches to curve number (`sq_daycn.f90`) or Green-Ampt (`sq_greenampt.f90`) for surface runoff.
- `swr_percmain.f90` is the master driver for layered soil percolation and lateral flow; it calls `swr_percmicro.f90`, `swr_percmacro.f90`, and `swr_satexcess.f90`.
- `swr_substor.f90` lags lateral and tile flow on the way to the channel.
- `sq_surfst.f90` applies `surlag` to delay surface runoff over more than one day.
- `et_pot.f90` and `et_act.f90` compute potential and actual evapotranspiration.
- `stor_surfstor.f90` updates surface storage from the depression-storage routine.
- `basin_output.f90`, `hru_output.f90`, and the `*_output.f90` family write the per-object water balance to `output.std` and the individual `*_wb` files.

Related pages

- Hydrology for surface runoff, lateral flow, percolation, and channel routing.
- Climate and Weather for the precipitation and PET inputs that drive the balance.
- Soil Water and Temperature for soil profile storage.
- Aquifers for what happens to `Q_perc` after it leaves the soil profile.
- `codes.bsn` and `parameters.bsn` for switches and coefficients.
- Water balance output for the corresponding output files.

Overview

SWAT+ drives the land phase with daily values of precipitation, maximum and minimum temperature, solar radiation, relative humidity, and wind speed. Each variable can come from a gauge file, from the weather generator, or from a mix of the two on a per-station basis. Potential evapotranspiration can be computed inside the model from three methods or read from a daily file.

Process equations

Inputs from gauge files

Each weather station listed in `weather-sta.cli` names up to six measured-data files: precipitation (`*.pcp`), temperature (`*.tmp`), solar radiation (`*.slr`), humidity (`*.hmd`), wind (`*.wnd`), and a precomputed PET file (`*.pet`). The strings `null` or a missing file fall back to weather-generator output for that variable. The master index of each measured variable across all stations is held in `pcp.cli`, `tmp.cli`, `slr.cli`, `hmd.cli`, `wnd.cli`, and `pet.cli`.

Weather generator

Stochastic weather is generated from monthly statistics held in `weather-wgn.cli` (one block per WGN station). Generation routines:

- Precipitation occurrence and amount: `cli_pgen.f90`, sub-daily disaggregation in `cli_pgenhr.f90`.
- Air temperature: `cli_tgen.f90`.
- Solar radiation: `cli_slrgen.f90`.
- Relative humidity: `cli_rhgen.f90`.
- Wind speed: `cli_wndgen.f90`.
- Driver and initialisation: `cli_clgen.f90`, `cli_initwgn.f90`, `cli_weatgn.f90`.

Optional elevation lapsing of precipitation and temperature is applied in `cli_lapse.f90` when `bsn_cc%lapse = 1`, using `bsn_prm%plaps` (mm/km, default 0.0) and `bsn_prm%tlaps` (deg C/km, default 6.5).

Potential evapotranspiration

The PET method is selected by `bsn_cc%pet` in `codes.bsn`. The dispatch is in `et_pot.f90`:

`bsn_cc%pet`	Method
0	Priestley-Taylor
1	Penman-Monteith
2	Hargreaves
3	Read daily PET from file

Priestley-Taylor (`case (0)` in `et_pot.f90`) uses

```
PET = alpha * (D / (D + g)) * Rn / lambda
```

with `alpha = 1.28`, slope of saturation vapour pressure curve `D`, psychrometric constant `g`, net radiation `Rn`, and latent heat of vaporization `lambda`. Net radiation is built from incoming solar radiation, albedo (with a snow adjustment when `sno_mm > 0.5 mm`), and a net long-wave term.

Penman-Monteith (`case (1)`) uses

```
PET = (D * Rn + rho * g * VPD / rv) / (lambda * (D + g * (1 + rc / rv)))
```

with aerodynamic resistance `rv`, canopy resistance `rc`, air density `rho`, and vapour pressure deficit `VPD`. For the reference PET, `rv = 114 / (u * (170/1000)**0.2)` and `rc = 49 / (1.4 - 0.4 * CO2 / 330)`. Maximum plant ET is computed separately using canopy-specific resistance values.

Hargreaves (`case (2)`) uses

```
PET = 0.0023 * (Ra / lambda) * (Tave + 17.8) * sqrt(Tmax - Tmin)
```

with extraterrestrial radiation `Ra = solradmx * 37.59 / 30`.

The read-in option (`case (3)`) takes the daily PET value from the gauge file pointed to by the station and applies no further computation in this routine. When `bsn_cc%pet == 3`, `pet.cli` is opened in `basin_read_cc.f90`.

A final scaling `pet_day = hru(j)%hyd%pet_co * pet_day` is applied at the bottom of `et_pot.f90`. The basin-level multiplier `bsn_prm%petco_pmpt` (default 100.0, percent) is used during calibration for Penman-Monteith and Priestley-Taylor.

Actual ET (canopy evaporation, transpiration, soil evaporation) is then partitioned in `et_act.f90` against `PET`, `esco`, and `epco`.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%pet`	0 (Priestley-Taylor)	`codes.bsn`	PET method
`bsn_cc%lapse`	0 (off)	`codes.bsn`	Apply elevation lapse to precipitation and temperature
`bsn_cc%petfile`	`pet.cli`	`codes.bsn`	Name of the PET index file when `pet = 3`
`bsn_prm%plaps`	0.0 mm/km	`parameters.bsn`	Precipitation lapse rate
`bsn_prm%tlaps`	6.5 deg C/km	`parameters.bsn`	Temperature lapse rate
`bsn_prm%petco_pmpt`	100.0 %	`parameters.bsn`	PET adjustment for Penman-Monteith and Priestley-Taylor
`bsn_prm%igen`	5	`parameters.bsn`	Random seed selection for the weather generator
`hyd_db%pet_co`	1.0	`hydrology.hyd`	Per-HRU PET multiplier

Implementation

Source modules in `swatplus/src/`:

- `cli_staread.f90` reads `weather-sta.cli` (the index of weather stations and their measured-data filenames).
- `cli_wgnread.f90` reads `weather-wgn.cli` (the index of WGN station parameter sets).
- `cli_pmeas.f90`, `cli_tmeas.f90`, `cli_smeas.f90`, `cli_hmeas.f90`, `cli_wmeas.f90`, `cli_petmeas.f90` read the measured `*.pcp`, `*.tmp`, `*.slr`, `*.hmd`, `*.wnd`, and `*.pet` files for each station.
- `cli_clgen.f90`, `cli_initwgn.f90`, `cli_weatgn.f90` drive the weather generator. Variable generators: `cli_pgen.f90`, `cli_tgen.f90`, `cli_slrgen.f90`, `cli_rhgen.f90`, `cli_wndgen.f90`.
- `cli_lapse.f90` applies elevation lapsing.
- `cli_precip_control.f90` coordinates daily and sub-daily precipitation steps.
- `et_pot.f90` computes potential ET (Priestley-Taylor, Penman-Monteith, Hargreaves, read).
- `et_act.f90` computes actual ET and partitions soil evaporation across layers.
- `basin_read_cc.f90` opens the PET index file when `bsn_cc%pet == 3`.

Related pages

- `codes.bsn` for PET and lapse switches.
- `parameters.bsn` for lapse rates and PET adjustment.
- `pcp`, `tmp`, `cli` for climate input file formats.
- Overview and Water Balance for how PET feeds the daily HRU balance.

Overview

The hydrology cluster is what moves water from precipitation through HRUs and along the channel network. Each HRU produces surface runoff, lateral subsurface flow, and percolation. Surface runoff and lateral flow lag through storage terms before reaching the receiving channel. Channels route the inflow to the outlet using one of two routing schemes selected by `bsn_cc%rte`.

Process equations

Surface runoff: curve number vs Green-Ampt

The infiltration / runoff partition is set by `bsn_cc%gampt` in `codes.bsn`. The dispatch lives in `sq_volq.f90`:

```
if (bsn_cc%gampt == 0) then
  call sq_daycn
else
  call sq_greenampt
end if
```

Curve number (`bsn_cc%gampt = 0`, daily time step). The SCS curve number method computed in `sq_daycn.f90`:

```
S = 25400 / CN - 254
Ia = 0.2 * S
Q_surf = (P - Ia)^2 / (P + 0.8 * S)   when P > Ia
       = 0                            otherwise
```

`CN` is updated daily for soil moisture in `sq_dailycn.f90`. The retention `S` shape is set per HRU by `hyd_db%cn3_swf` (soil water at curve number condition III, where 0 = field capacity and 0.99 = near saturation). A separate frozen-soil adjustment uses `bsn_prm%cn_froz` (default 0.000862). For urban HRUs, an impervious-area curve number of 98 is mixed by the impervious fraction `fcimp`.

Green-Ampt (`bsn_cc%gampt = 1`, sub-daily). Used when sub-daily precipitation is available. The infiltration rate at cumulative infiltration `F` is

```
f = K * (1 + psi * dtheta / F)
```

with hydraulic conductivity `K`, wetting-front suction `psi`, and initial moisture deficit `dtheta`. The implementation iterates on cumulative infiltration each sub-daily time step in `sq_greenampt.f90`. Initial abstraction for urban HRUs is capped at `bsn_prm%urb_init_abst` (default 1.0 mm).

Surface runoff lag. Generated runoff `Q_surf` is held by `sq_surfst.f90` so that only a fraction reaches the channel on the day of the event:

```
brt = 1 - exp(-surlag / t_conc)
qday = (lagged_storage + Q_surf) * brt
```

`bsn_prm%surlag` (default 4.0 days) and the HRU time of concentration `t_conc` together set `brt`.

Lateral subsurface flow

Lateral flow is computed layer by layer in `swr_percmicro.f90` from the gravity-drained water above field capacity. Each layer produces a lateral component `latlyr` and a vertical percolation component `sepday`. The layer values are summed into `latq(j)` in `swr_percmain.f90`:

```
latq(j) = latq(j) + latlyr
```

Lateral flow is then lagged across the basin via `swr_substor.f90`:

```
latq_out = lagged_storage * lat_ttime
```

where `hyd_db%lat_ttime` is the lateral travel-time factor read from `hydrology.hyd`. A linear adjustment `hyd_db%latq_co` (default 0.3) scales daily lateral flow.

Percolation and return flow

The master driver `swr_percmain.f90` walks the soil profile from top to bottom. For each layer:

1. Add gravity-drained water from the overlying layer.
2. Compute `sw_excess = st - fc` (storage above field capacity).
3. Compute layer percolation `sepday` and lateral flow `latlyr` using a storage routing formula with characteristic time `TT = (sat - fc) / K_sat`. The implementation is in `swr_percmicro.f90`; macropore flow is handled in `swr_percmacro.f90`.
4. Subtract `sepday`, `latlyr`, and tile flow from layer storage.

Percolation from the bottom of the soil profile (`sepbtm`) recharges the shallow aquifer. Return flow (baseflow back to the channel) is computed in the aquifer module from the shallow aquifer storage. See Aquifers.

Tile drainage from the layer is computed with `swr_drains.f90` (DRAINMOD equations, `bsn_cc%tdrn = 1`) or `swr_origtile.f90` (drawdown-days equation, `bsn_cc%tdrn = 0`). Saturated profile redistribution sits in `swr_satexcess.f90`.

Channel routing: variable storage vs Muskingum

`bsn_cc%rte` selects the channel routing method:

`bsn_cc%rte`	Method
0	Variable storage coefficient
1	Muskingum

Both methods run in `ch_rtmusk.f90` (it dispatches on `bsn_cc%rte` inside its sub-daily loop, despite the file name).

Variable storage. Outflow at the sub-step is

```
scoef = bsn_prm%scoef * 2 * dt / (2 * ttime + dt)
outflow = scoef * tot_storage
```

The travel time `ttime` is interpolated from the rating curve at the current inflow and outflow. `bsn_prm%scoef` (default 1.0) is the channel storage coefficient.

Muskingum. The classical three-coefficient form:

```
O2 = c1 * I2 + c2 * I1 + c3 * O1
```

with `c1`, `c2`, `c3` derived from the Muskingum storage time constant `Km` and weighting factor `X`. `bsn_prm%msk_co1` (default 0.75), `bsn_prm%msk_co2` (default 0.25), and `bsn_prm%msk_x` (default 0.20) control the calibration of `Km` from the bankfull and low-flow reach response times.

Transmission losses are subtracted after routing using the channel bed hydraulic conductivity `sd_ch%chk` over the reach length and wetted perimeter. Reach evaporation uses `bsn_prm%evrch` (default 0.60).

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%gampt`	0	codes.bsn	0 = curve number; 1 = Green-Ampt
`bsn_cc%rte`	0	codes.bsn	0 = variable storage; 1 = Muskingum
`bsn_cc%tdrn`	0	codes.bsn	0 = drawdown-days tile flow; 1 = DRAINMOD
`bsn_cc%wtdn`	0	codes.bsn	Shallow water table method
`bsn_cc%crk`	0	codes.bsn	1 = compute crack flow

Switch / parameter	Default	File	Effect
bsn_cc%sed_det	0	codes.bsn	Peak rate method (NRCS UH vs half-hour rainfall)
bsn_prm%surlag	4.0 days	parameters.bsn	Surface runoff lag
bsn_prm%scoef	1.0	parameters.bsn	Variable-storage channel coefficient
bsn_prm%msk_co1	0.75	parameters.bsn	Muskingum bankfull storage coefficient
bsn_prm%msk_co2	0.25	parameters.bsn	Muskingum low-flow storage coefficient
bsn_prm%msk_x	0.20	parameters.bsn	Muskingum weighting factor
bsn_prm%evrch	0.60	parameters.bsn	Reach evaporation adjustment
bsn_prm%cn_froz	0.000862	parameters.bsn	Frozen soil CN adjustment
bsn_prm%urb_init_abst	1.0 mm	parameters.bsn	Urban max initial abstraction (Green-Ampt)
bsn_prm%adj_pkr	1.0	parameters.bsn	Subbasin peak rate adjustment
bsn_prm%prf	484.0	parameters.bsn	Peak rate factor for peak rate equation
bsn_prm%uhalpha	1.0	parameters.bsn	Gamma-UH alpha coefficient
hyd_db%cn3_swf	per HRU	hydrology.hyd	Soil water at CN3
hyd_db%lat_ttime	per HRU	hydrology.hyd	Lateral flow travel time
hyd_db%latq_co	0.3	hydrology.hyd	Lateral flow linear adjustment
hyd_db%perco	per HRU	hydrology.hyd	Percolation linear adjustment

Implementation

Source modules in `swatplus/src/`:

- `sq_volq.f90` dispatches surface runoff calculation.
- `sq_daycn.f90` computes daily curve-number runoff; `sq_dailycn.f90` adjusts CN for soil moisture each day.
- `sq_greenampt.f90` computes sub-daily Green-Ampt infiltration and runoff.
- `sq_surfst.f90` applies the surface runoff lag.
- `sq_canopyint.f90` removes canopy interception before infiltration.
- `swr_percmain.f90` is the percolation master; `swr_percmicro.f90` handles micropore flow, `swr_percmacro.f90` macropore flow.
- `swr_substor.f90` lags lateral and tile flow.
- `swr_satexcess.f90` redistributes water above saturation.
- `swr_drains.f90` and `swr_origtile.f90` compute tile flow.
- `ch_rtday.f90` routes daily channel flow (older path).
- `ch_rtmusk.f90` runs the sub-daily Muskingum and variable-storage routing.
- `sd_channel_control3.f90` is the master channel control routine.
- `cn2_init.f90` and `cn2_init_all.f90` initialise the curve number per HRU.

Related pages

- codes.bsn for routing and runoff switches.
- parameters.bsn for routing coefficients.
- hru for HRU pointers to `hydrology.hyd`.
- Aquifers for what happens to percolation after it leaves the soil.
- Channels for channel geometry and rating curves.

- Soil Water and Temperature for the soil profile representation.
- Water balance output.

Overview

Every HRU has a discretised soil profile of up to ten layers. Each layer holds physical properties (depth, bulk density, hydraulic conductivity, sand-silt-clay-rock fractions, organic content) and state variables (water storage, temperature, layer chemistry). The soil profile is the central reservoir for the HRU water balance: surface runoff is partitioned at the top of layer 1, percolation drains the bottom, and lateral and tile flow leave each layer between. Soil temperature is updated daily for each layer and is used to gate decomposition, nitrification, and biological processes.

Process equations

Soil layers

The layer geometry is held in `soil_module.f90` (`type soil_profile`). Each layer carries:

- `d` (mm) cumulative depth from surface
- `thick` (mm) layer thickness
- `bd` (Mg/m^3) bulk density (default 1.3, bounded to 2.0)
- `k` (mm/hr) saturated hydraulic conductivity (defaulted by hydrologic group A=50, B=20, C=5, D=2 when missing)
- `awc` (mm/mm) available water capacity, bounded to `[0.005, 0.8]`
- `clay`, `silt`, `sand`, `rock` fractions
- `wp`, `up`, `por`, `fc`, `st` water content terms (see below)

Initialisation lives in `soil_phys_init.f90`. Defaults applied there:

```
wp  = 0.4 * clay * bd / 100          ! wilting point (m^3/m^3)
up  = wp + awc                       ! upper limit (field capacity, m^3/m^3)
por = 1 - bd / 2.65                  ! porosity from bulk density
fc  = thick * (up - wp)              ! field capacity water in mm
st  = fc * ffc                       ! initial layer water as fraction of FC
```

The basin-wide initial soil water fraction `bsn_prm%ffcb` (default 0.0) seeds layer storage at the start of the run.

Available water capacity

The available water capacity of a layer is the water held between field capacity and wilting point:

```
AWC_layer (mm) = thick * (up - wp) = thick * awc
```

The profile sum is `soil(j)%s%sumfc`. This drives plant water uptake (compared against `epco`) and soil evaporation (compared against `esco`).

Daily soil water balance

For each layer, the daily update is

```
st(t+1) = st(t) + inflow - percolation - lateral - tile - soil_evap - transpiration
```

Inflow to layer 1 is infiltration `P - Q_surf`. Inflow to deeper layers is percolation from the layer above. Gravity drainage is allowed when `st > fc`. The implementation runs in `swr_percmain.f90` and `swr_percmicro.f90` (see Hydrology).

Soil temperature

Soil temperature at the centre of each layer is computed daily in `stmp_solt.f90` from the soil surface temperature, an annual mean air temperature, and a depth-dependent damping function.

Maximum damping depth (mm), from bulk density:

```
f  = avbd / (avbd + 686 * exp(-5.63 * avbd))
dp = 1000 + 2500 * f
```

Soil water scaling factor:

```
ww = 0.356 - 0.144 * avbd
wc = sw / (ww * d_bottom)
```

Daily damping depth:

```
b  = ln(500 / dp)
dd = dp * exp(b * ((1 - wc) / (1 + wc))^2)
```

Bare soil surface temperature:

```
st0   = (solrad * (1 - albedo) - 14) / 20
tbare = tave + 0.5 * (tmax - tmin) * st0
```

Cover lagging factor (residue, biomass, snow):

```
cover = above_ground_mass + residue_mass
bcv   = cover / (cover + exp(7.563 - 1.297e-4 * cover))
```

If `sno_mm > 0`, `bcv` is taken as the maximum of the residue and snow lagging factors. Surface temperature with cover:

```
tcov           = bcv * tmp_layer2 + (1 - bcv) * tbare
soil%tmp_srf   = 0.5 * (tbare + tcov)
```

Layer temperature with the lag coefficient `tlag = 0.8`:

```
zd  = depth_center / dd
df  = zd / (zd + exp(-0.8669 - 2.0775 * zd))
tmp = tlag * tmp_prev + (1 - tlag) * (df * (tmp_an - tmp_srf) + tmp_srf)
```

where `tmp_an` is the long-term annual mean air temperature for the WGN station. Layer temperature feeds frozen-soil flags, decomposition rates, and septic system temperature corrections.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_prm%ffcb`	0.0	`parameters.bsn`	Initial soil water as fraction of field capacity
`bsn_prm%cn_froz`	0.000862	`parameters.bsn`	Frozen soil CN adjustment for infiltration
`bsn_cc%wtdn`	0	`codes.bsn`	Shallow water table depth algorithm
`hyd_db%esco`	per HRU	`hydrology.hyd`	Soil evaporation compensation factor (0-1)
`hyd_db%epco`	per HRU	`hydrology.hyd`	Plant water uptake compensation factor (0-1)
`hyd_db%canmx`	per HRU	`hydrology.hyd`	Maximum canopy storage
Layer fields `bd`, `awc`, `k`, `clay`, `silt`, `sand`, `rock`, `cbn`	per layer	`sol`	Soil layer physical properties

Implementation

Source modules in `swatplus/src/`:

- `soil_module.f90` defines the `soil_profile` derived type and per-layer physical and chemical fields.
- `soil_phys_init.f90` initialises layer water-retention parameters (wp, up, por, fc, st) and applies the hydrologic-group default conductivities.
- `solt_db_read.f90` reads the soil temperature lookup data.
- `swr_percmain.f90` updates layer water storage through the daily percolation pass.
- `swr_satexcess.f90` redistributes water above saturation.
- `stmp_solt.f90` computes daily soil layer temperatures.
- `basin_sw_init.f90` initialises soil water using `ffcb` at simulation start.
- `et_act.f90` removes soil evaporation from layers using `esco`.
- `albedo.f90` updates surface albedo, which feeds the bare soil temperature.

Related pages

- `sol` for the soil layer input file.
- Hydrology for percolation and lateral flow that change layer storage.
- Plant growth and land use management for transpiration, which removes layer water.
- Nutrient cycling for layer chemistry and temperature dependence of mineralisation.
- Carbon for the Century-style organic matter pools driven by soil temperature.

Overview

Soil loss at the HRU is computed with the Modified Universal Soil Loss Equation (MUSLE). The HRU sediment yield is then routed through the channel network, where each reach can erode its bed and banks or deposit suspended sediment. The peak runoff rate that drives MUSLE comes from one of two methods set by `bsn_cc%sed_det`. Channel sediment processes use the rating curve and reach geometry from the SWAT+ degradation channel.

Process equations

MUSLE for HRU sediment yield

The MUSLE form used in `ero_ysed.f90`:

```
sed = 10 * (Q_surf * q_peak * A_ha) ^ 0.56 * cklsp
```

Symbol	Meaning
`sed`	Daily sediment yield (t)
`Q_surf`	Daily surface runoff volume (mm)
`q_peak`	Peak runoff rate (m^3/s)
`A_ha`	HRU area (ha)
`cklsp`	Combined cover (C), management (P), topographic (LS), and soil erodibility (K) factor

`cklsp` is built per HRU as `cklsp = usle_cfac * usle_mult`, where `usle_mult = K * LS * P * 11.8` is set up at HRU initialisation and `usle_cfac` is the dynamic cover factor from `ero_cfactor.f90`.

The sediment yield is reduced under snow cover using:

```
sed = sed / exp(3 * sno_mm / 25.4)
```

and is set to zero when the snowpack exceeds 100 mm.

For reference, USLE itself is computed alongside for output as `usle = 1.292 * usle_ei * cklsp / 11.8`, where `usle_ei` is the rainfall erosivity from `ero_eiusle.f90`.

Peak runoff rate

The peak flow `q_peak` is selected by `bsn_cc%sed_det` in `ero_pkq.f90`:

- `bsn_cc%sed_det = 0` (default): NRCS dimensionless unit hydrograph with peak rate factor `bsn_prm%prf` (default 484.0).
- `bsn_cc%sed_det = 1`: half-hour rainfall intensity method using `precip_half_hr` from the climate or WGN data.

NRCS form (default):

```
q_peak (cfs) = prf / 6578.6 * A_ha * qday / tconc
q_peak (cms) = q_peak (cfs) / 35.3
```

Half-hour form:

```
altc      = 1 - exp(2 * tconc * ln(1 - alpha_half))
q_peak    = altc * qday / tconc * A_km^2 / 3.6   (m^3/s)
```

with `alpha_half = precip_half_hr` as a fraction of daily rainfall.

`bsn_prm%adj_pkr` (default 1.0) is the subbasin peak rate adjustment factor.

Sediment in lateral and tile flow

`swr_latsed.f90` adds a sediment load to the HRU yield from lateral and tile flow:

```
sed_lat = lat_sed * (Q_lat + Q_tile) * A_ha / 100000
```

with `hyd_db%lat_sed` (g/L) read from `hydrology.hyd`. Particle size split (sand, silt, clay, small aggregate, large aggregate) follows the soil texture detachment fractions.

Overland sediment routing

`ero_ovrsed.f90` partitions HRU sediment yield by particle size class and applies an enrichment ratio. When `bsn_cc%gampt > 0` (sub-daily simulations), the routine also distributes sediment in time using the sub-daily runoff pattern.

Channel sediment transport

Channel sediment processes run in `sd_channel_sediment3.f90` (the active routine for SWAT+ degradation channels). Two erosion sources and one deposition sink are computed each day.

Critical velocity for bank erosion is built from a clay cohesion term and a vegetation cover term:

```
cohesion  = -87.1 + 42.82 * ch_clay - 0.261 * ch_clay^2 + 0.029 * ch_clay^3
veg       = exp(-5 * chd) * 4500 * cov
bd_fac    = max(0.001, 0.03924 * ch_bd * 1000 - 1000)
cohes_fac = 0.021 * cohesion + veg
v_cr      = vcr_coef * log10(2200 * chd) * sqrt(0.0004 * (bd_fac + cohes_fac))
```

with `ch_clay` (channel clay percent), `ch_bd` (channel bed bulk density), `chd` (channel depth), and `cov` (channel cover). The mean reach velocity adjusted for meander curvature is

```
rad_curv = (12 * chw) * sinu^1.5 / (13 * (sinu - 1)^0.5)
v_bend   = v * (1 / rad_curv + 1)
v_rch    = 0.33 * v_bend + 0.66 * v
```

Bank erosion is computed when `v_rch > v_cr`:

```
ebank_m = 1 / (1 / v_vc + 1 / m_exhaust)
ebank_t = 1000 * ebank_m * chd * arc_len * ch_bd
```

with `arc_len = 0.25 * chl` and `m_exhaust = 0.0002 * chw`.

Bed erosion is allowed when channel slope exceeds an equilibrium slope. Critical shear comes from the median bed particle `d50`. Acting shear is

```
shear_btm = 9800 * dep * chs   (Pa)
```

with `dep` the flow depth and `chs` the channel slope.

Channel deposition is taken as a bedload fraction of bank erosion:

```
ch_dep_sed = wash_bed_fr * bank_ero_sed
```

Suspended sediment in the routed hydrograph carries associated organic N and P using channel-specific concentrations (`n_conc`, `p_conc`) and enrichment factors (`n_dep_enr`, `p_dep_enr`).

Note: the legacy `bsn_prm%spcon` and `bsn_prm%spexp` parameters are listed in `basin_module.f90` as "not used". The active sediment routing in `sd_channel_sediment3.f90` is based on the velocity and shear formulation above, not on a sediment transport capacity power law.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%sed_det`	0	`codes.bsn`	0 = NRCS UH peak rate, 1 = half-hour intensity
`bsn_cc%gampt`	0	`codes.bsn`	Triggers sub-daily sediment distribution when > 0
`bsn_prm%prf`	484.0	`parameters.bsn`	Peak rate factor for the NRCS method
`bsn_prm%adj_pkr`	1.0	`parameters.bsn`	Subbasin peak rate adjustment
`bsn_prm%eros_spl`	0.0	`parameters.bsn`	Splash erosion coefficient (0.9-3.1)
`bsn_prm%rill_mult`	0.0	`parameters.bsn`	Rill erosion coefficient
`bsn_prm%eros_expo`	0.0	`parameters.bsn`	Exponent for overland flow erosion
`bsn_prm%c_factor`	0.0	`parameters.bsn`	Scaling for overland C-factor
`bsn_prm%ch_d50`	0.0	`parameters.bsn`	Median particle diameter of main channel (mm)
`hyd_db%lat_sed`	per HRU	`hydrology.hyd`	Sediment concentration in lateral flow (g/L)
`hyd_db%erorgn`, `hyd_db%erorgp`	per HRU	`hydrology.hyd`	Organic N and P enrichment ratios
`sd_ch%ch_clay`, `ch_bd`, `cov`, `d50`, `vcr_coef`, `wash_bed_fr`	per channel	`sd-channel`	Channel sediment properties

Implementation

Source modules in `swatplus/src/`:

- `ero_ysed.f90` implements MUSLE for HRU daily sediment yield.
- `ero_pkq.f90` computes the peak runoff rate used by MUSLE.
- `ero_cfactor.f90` updates the dynamic cover and management factor.
- `ero_eiusle.f90` computes rainfall erosivity for the reference USLE.
- `ero_ovrsed.f90` distributes sediment in time and by particle size class for sub-daily simulations.
- `swr_latsed.f90` adds sediment from lateral and tile flow to the HRU yield.
- `sd_channel_sediment3.f90` is the active channel sediment routine: bank erosion, bed erosion, deposition, and associated nutrients.
- `sd_channel_control3.f90` is the channel master that calls the sediment routine.

- `nut_psed.f90` handles sediment-bound phosphorus on land.

Related pages

- Hydrology for surface runoff and peak rate.
- Channels for reach geometry, rating curve, and routing.
- `sd-channel` for channel sediment properties.
- `parameters.bsn` for basin-level erosion coefficients.
- Channel sediment output for the corresponding output.

Overview

SWAT+ grows one or more plants in a community on each HRU. Daily potential biomass is a function of intercepted photosynthetically active radiation. That potential is reduced by water, temperature, nitrogen, phosphorus, and aeration stresses. Phenology is tracked by accumulated plant heat units (PHU). The plants on an HRU are scheduled by `management.sch`, and land use plus structural BMPs are assigned through `landuse.lum`. Decision tables (`lum.dtl`) drive conditional operations such as auto-irrigation, auto-fertilisation, and land use change.

Process equations

Heat unit accumulation

Each plant accumulates a fraction of total heat units to maturity per day:

```
delg   = max(0, (tave - t_base) / phumat)
phuacc = phuacc + delg
```

`t_base` is the plant's minimum growth temperature (`plants.plt`), `phumat` is the heat units to maturity. When `phuacc` reaches 1.0 the plant is mature. `phuacc_p` is the perennial counterpart. Source: `pl_nut_demand.f90`.

Biomass accumulation

Daily potential biomass is

```
bioday = beadj * par
```

where `par` is intercepted PAR (MJ/m^2) and `beadj` is radiation use efficiency adjusted for CO2 and vapour pressure deficit. The CO2 adjustment uses the two-point curve described by `bio_e`, `bioehi`, `co2hi` in `plants.plt`. When vapour pressure deficit exceeds 1.0 kPa, RUE declines at the plant-specific rate `wavp`, with a floor of `0.27 * bio_e`. Realised biomass is

```
bm_grow = bioday * reg
reg     = min(strsw, strst, strsn, strsp, strsa[, strss])
```

`reg` is the most limiting stress factor. Plant carbon mass is fixed at `0.42 * biomass`. Source: `pl_biomass_gro.f90`.

Leaf area index

LAI follows a two-point optimal curve defined by `(frgrw1, laimx1)` and `(frgrw2, laimx2)` from `plants.plt`. The model computes a fraction `f` of maximum LAI from `phuacc`:

```
f = phuacc / (phuacc + exp(leaf1 - leaf2 * phuacc))
```

`leaf1` and `leaf2` are shape parameters fitted from the two points. Before `phuacc` reaches `dlai` (fraction of season at which leaves decline), LAI grows toward `laimax`:

```
deltaLAI = (f - laimxfr) * laimax * (1 - exp(5*(LAI - laimax))) * sqrt(strsw)
```

After `phuacc > dlai`, LAI declines following `dlai_rate`. Canopy height is `chtmx * sqrt(f)`. For perennials, LAI is scaled by tree age through `laixco_tree` and `mat_yrs`. Source: `pl_leaf_gro.f90`, `pl_leaf_senes.f90`.

Water stress and uptake

Potential transpiration `epmax(ipl)` is distributed through the rooting depth using an exponential extraction function with parameters `uptake%water_dis` and `uptake%water_norm`. Water uptake from each layer is

```
sum = epmax * (1 - exp(-water_dis * gx / root_dep)) / water_norm
wuse = sum - sump * (1 - epco)
```

where `gx` is depth to the bottom of the current layer. Layers below the root zone do not contribute. The compensation factor `epco` (0-1) controls how much a deeper layer can compensate when a shallower layer is dry. Water stress is

```
strsw = sum_wuse / epmax
```

Aeration stress activates when soil water exceeds field capacity:

```
satco = (sw - sumfc) / (sumul - sumfc)
strsa = 1 - scparm / (scparm + exp(2.9014 - 0.03867*scparm))
```

where `scparm = 100 * (satco - aeration) / (1.0001 - aeration)`. Source: `pl_waterup.f90`.

Temperature stress

```
rto   = (tave - t_base) / (t_opt - t_base)
strst = sin(pi/2 * rto)
```

`strst = 0` if `tave <= t_base` or if `tmin <= annual mean - 15`. Source: `pl_tstr.f90`.

Nitrogen and phosphorus stress and uptake

Plant N and P demands follow optimal concentration curves with `pltnfr1`, `pltnfr2`, `pltnfr3` and `pltpfr1`, `pltpfr2`, `pltpfr3` (see `plants.plt`). N and P are taken from soil layers with a depth-distribution function controlled by `n_updis` and `p_updis` in `parameters.bsn`. If supply is below demand, `strsn` and `strsp` reduce growth. Legumes fix N when soil supply is short:

```
fxr   = min(1, fxw, fxn) * fxg
fixn  = nfix_co * fxr * deficit + (1 - nfix_co) * deficit
fixn  = min(nfixmx, fixn)
```

`nfix_co` is the legume coefficient (0.5 for legumes, 0 for non-legumes). `nfixmx` (default 20 kg N/ha/day, `parameters.bsn`) caps daily fixation. Source: `pl_nup.f90`, `pl_pup.f90`, `pl_nfix.f90`, `pl_nupd.f90`, `pl_pupd.f90`.

Harvest

Yield is `harveff * seed_mass`. The harvest index is bounded below by `wsyf` and accumulates from zero up to `hvsti` as `phuacc` increases past 0.5. Harvest types in `harv.ops` are `grain`, `biomass`, `residue`, `tree`, `tuber`, `peanuts`, `stripper`, `picker`. Pesticide is removed in yield in proportion to `pl_yield%m / pl_mass(j)%tot(ipl)%m`. Source: `mgt_harvgrain.f90`, `mgt_harvbiomass.f90`, `mgt_harvresidue.f90`, `mgt_harvtuber.f90`, `mgt_killop.f90`.

Land use management

`landuse.lum`

Each HRU points to a row in `landuse.lum`. That row carries the plant community name, curve number group, USLE conservation practice, tillage practice, and pointers to the management schedule, tile drainage code, septic code, filter strip, grass waterway, and BMP user practice. `hru_lum_init.f90` copies these pointers onto each HRU at the start of simulation. Source: `hru_lum_init.f90`, `landuse_data_module.f90`.

`management.sch`

For each schedule, operations are listed in calendar or PHU-trigger order. Operation codes used by `mgt_sched.f90` include:

Code	Operation
`plnt`	Plant a single species (or community)
`harv`	Harvest (grain / biomass / residue / tree / tuber etc.)
`kill`	Kill the plant, transfer all biomass to residue
`hvkl`	Harvest and kill in the same operation
`till`	Tillage, mixes residue and pools using `till.til`
`irrm`	Irrigation, applies depth from `irr.ops`
`fert`	Mineral or organic fertiliser (`fertilizer.frt`)
`manu`	Manure application
`pest`	Pesticide application
`graz`	Grazing
`cnup`	Curve number update
`burn`	Prescribed burn
`swep`	Street sweeping
`dwm`	Drainage water management depth
`weir`	Set or adjust weir height
`mons`	Begin or end monsoon trigger window

PHU-triggered operations fire when `phuacc` of the plant in column `op2` crosses the value in column `phu`. Date-triggered operations fire on the given month and day. Source: `mgt_sched.f90`, `mgt_operations_module.f90`.

Decision tables (`lum.dtl`)

Each row in `landuse.lum` may also reference one or more decision tables in `lum.dtl`. Decision tables are conditional rule sets that fire actions when their conditions are met. Each table has `conds` conditions, `alts` alternative outcomes, and `acts` actions. On each day in `hru_control.f90` the model loops over the HRU's `num_autos` auto operations and calls `conditions` then `actions` for each. Typical actions include auto-irrigation, auto-fertilisation, planting based on PHU triggers, and land use change. The decision table reader is `dtbl_lum_read.f90`; the run-time logic is in `conditions.f90` and `actions.f90`.

See the Decision tables theory page for the rule grammar.

Switches and parameters

File	Field	Effect
`codes.bsn`	`nostress`	0 = all stresses applied, 1 = no stress, 2 = no N/P stress
`codes.bsn`	`cswat`	Selects soil carbon model. The dynamic CENTURY/SWAT-C model (=2) changes residue and tillage routines
`parameters.bsn`	`co2`	Ambient CO2 concentration (ppm), default 400
`parameters.bsn`	`nfixmx`	Maximum daily N fixation (kg/ha), default 20
`parameters.bsn`	`n_updis`, `p_updis`	Depth-distribution shape for plant N/P uptake
`parameters.bsn`	`evlai`	LAI above which only transpiration occurs, default 3.0
`plants.plt`	`bio_e`, `bioehi`, `co2hi`, `wavp`	RUE and its CO2 / VPD response
`plants.plt`	`blai`, `frgrw1`, `laimx1`, `frgrw2`, `laimx2`, `dlai`, `dlai_rate`	LAI development curve
`plants.plt`	`t_base`, `t_opt`	Temperature response
`plants.plt`	`hvsti`, `wsyf`	Harvest index and water-stress floor
`plants.plt`	`chtmx`, `rdmx`	Maximum canopy height and root depth
`plants.plt`	`aeration`	Aeration stress threshold (fraction above FC)
`plants.plt`	`pltnfr1..3`, `pltpfr1..3`	N and P optimal concentration curves

File	Field	Effect
plants.plt	nfix_co	Legume coefficient
plants.plt	mat_yrs, laixco_tree, bmx_peren, leaf_tov_min, leaf_tov_max	Perennial / tree parameters

Implementation

Plant growth: pl_grow.f90, pl_biomass_gro.f90, pl_leaf_gro.f90, pl_leaf_senes.f90, pl_root_gro.f90, pl_seed_gro.f90, pl_partition.f90, pl_community.f90, pl_waterup.f90, pl_tstr.f90, pl_nut_demand.f90, pl_nup.f90, pl_pup.f90, pl_nupd.f90, pl_pupd.f90, pl_nfix.f90, pl_dormant.f90, pl_mortality.f90, pl_rootfr.f90.

Plant data and initialisation: plant_data_module.f90, plant_module.f90, plant_parm_read.f90, plant_init.f90, plant_all_init.f90, plant_transplant_read.f90.

Management operations: mgt_sched.f90, mgt_operatn.f90, mgt_plantop.f90, mgt_harvgrain.f90, mgt_harvbiomass.f90, mgt_harvresidue.f90, mgt_harvtuber.f90, mgt_killop.f90, mgt_biomix.f90, mgt_newtillmix_cswat0.f90, mgt_newtillmix_cswat1.f90, mgt_newtillmix_wet.f90, mgt_tillfactor.f90, mgt_transplant.f90, mgt_operations_module.f90, mgt_read_*.f90.

Plant operation utilities: pl_fert.f90, pl_fert_wet.f90, pl_manure.f90, pl_graze.f90, pl_burnop.f90.

Land use: hru_lum_init.f90, hru_lum_init_all.f90, landuse_data_module.f90, lum_read*.f90.

Decision tables: dtbl_lum_read.f90, conditions.f90, actions.f90, conditional_module.f90.

Related pages

- landuse.lum
- plants.plt
- fertilizer.frt
- management.sch
- lum.dtl
- codes.bsn
- parameters.bsn
- Decision tables
- Nutrient cycling
- Carbon

Overview

Nitrogen and phosphorus in SWAT+ move among organic, mineral, and plant pools in each soil layer and are transported to channels by surface runoff, lateral flow, tile drainage, and percolation. Once in a channel they are transformed by the QUAL2E in-stream water quality routines. The legacy soil P model (one stable pool) and the Vadas and White (2010) model (multiple pools) are selectable through bsn_cc%sol_p_model.

Process equations

Soil pools

Each soil layer carries the following N pools:

- nitrate mn(k)%no3
- ammonium mn(k)%nh4
- active organic humus N hact(k)%n
- stable organic humus N hsta(k)%n
- fresh organic residue N (per plant) pl(ipl)%rsd(k)%n

P pools per layer:

- labile mineral `mp(k)%lab`
- active mineral `mp(k)%act`
- stable mineral `mp(k)%sta`
- humus organic P `hsta(k)%p`
- fresh residue P `pl(ipl)%rsd(k)%p`

Combined temperature and moisture factor

Most rates are scaled by a combined factor `csf = sqrt(cdg * sut)` where

```
sut = max(0.05, 0.1 + 0.9 * sqrt(st/fc))
cdg = max(0.1, 0.9 * tmp / (tmp + exp(9.93 - 0.312*tmp)) + 0.1)
```

`st` is soil water, `fc` field capacity, `tmp` layer temperature. Mineralisation only occurs when `tmp > 0`. Source: `nut_nminrl.f90`.

Humus mineralisation

A fixed fraction `nactfr = 0.02` of organic N is held in the active pool. Flow between the active and stable pools maintains that fraction:

```
rwn = 1e-5 * (hact*n * (1/nactfr - 1) - hsta*n)
```

Active humus N mineralises to nitrate:

```
hmn = cmn * csf * hact%n
```

with `cmn` from `parameters.bsn` (default 0.003). Mineralised P from humus:

```
hmp = 1.4 * hmn * hsta%p / (hsta%n + hact%n)
```

Source: `nut_nminrl.f90`.

Residue decomposition

For each plant residue pool in each layer:

```
cnr  = rsd%c / rsd%n              # C:N ratio (capped at 500)
cnrf = exp(-0.693 * (cnr - 25)/25)
cpr  = rsd%c / rsd%p              # C:P ratio (capped at 5000)
cprf = exp(-0.693 * (cpr - 200)/200)
ca   = min(cnrf, cprf, 1)
decr = max(decr_min, min(1, rsdco_pl * ca * csf))
decomp = decr * rsd
```

80% of decomposed residue N and P goes to mineral pools (NO3 and labile P), 20% to humus. `rsdco_pl` is per plant (`plants.plt`); `decr_min` is in `parameters.bsn`. Source: `nut_nminrl.f90`, `cbn_rsd_decomp.f90`.

Nitrification and ammonia volatilisation

Computed in `nut_nitvol.f90`. Temperature factor `tf = 0.41 * (tmp - 5) / 10` activates when `tf >= 0.001`. The water factor `swf` ramps from 0 at wilting point to 1 at field capacity + 25%. The depth factor `dpf = 1 - dmid/(dmid + exp(4.706 - 0.0305*dmid))` increases nitrification with depth and decreases volatilisation. Combined coefficients:

```
akn = tf * swf                 # nitrification
akv = tf * dpf * cecf          # volatilisation (cecf = 0.15)
rnv = nh4 * (1 - exp(-akn - akv))
rnit = rnv * (1 - exp(-akn)) / [(1 - exp(-akn)) + (1 - exp(-akv))]
rvol = rnv - rnit
```

NO3 increases by `rnit`, NH4 decreases by `rnit + rvol`.

Denitrification

In `nut_nminrl.f90`:

```
if (sut >= sdnco):
    wdn = no3 * (1 - exp(-cdn * cdg * cbn/100))
```

where `cbn` is layer organic carbon (%), `cdn` is the exponential rate coefficient and `sdnco` is the saturation threshold, both in `parameters.bsn`. Defaults: `cdn = 1.40`, `sdnco = 1.30`.

Nitrate leaching and surface transport

In `nut_nlch.f90`, the layer mixes nitrate with mobile water:

```
vv = perc + flat + (surfq if layer 1) + (qtile if tile layer)
co = no3 * (1 - exp(-vv / (porosity_term))) / vv
```

Surface runoff carries `nperco * co * surfq`, lateral flow carries `co * flat`, tile flow uses `nperco_lchtile * co`, and the remainder percolates to the next layer. `nperco` and `nperco_lchtile` come from `parameters.bsn` (defaults 0.10 and 0.50).

Organic N and P transport with sediment

`nut_orgn.f90` and `nut_orgnc.f90` compute sediment-bound organic N. Concentration in the surface 10 mm is multiplied by `sedyld` and an enrichment ratio. The default enrichment ratio uses the CREAMS form

```
enratio = 0.78 * (0.1 * sedyld / (area * surfq)) ^ -0.2468
```

clipped to 3.0 (`pest_enrsb.f90`). Sediment-attached labile P comes from `nut_psed.f90`.

Soluble P loss and leaching

`nut_solp.f90` partitions surface labile P between water and soil via `phoskd` (`parameters.bsn`, default 175):

```
surqsolp = lab * surfq / (bd * d * phoskd + 1)
```

Leaching uses a similar partition with `pperco` (default 10.0):

```
vap  = -prk / (0.01 * st + 0.1 * pperco * bd)
plch = 0.001 * lab * (1 - exp(vap))
```

Mineral P transformations

`nut_pminrl.f90` (legacy model, `sol_p_model = 0`) flows P among labile, active, and stable pools to maintain the ratio `psp / (1 - psp)` between labile and active. Slow active-to-stable flow uses `bk = 0.01`. `nut_pminrl2.f90` is the Vadas and White (2010) reformulation, selected by `sol_p_model = 1`.

Plant N fixation

See plant growth. Fixation only kicks in when soil supply is below demand and the plant has `nfix_co > 0`. Capped daily by `nfixmx` (`parameters.bsn`, default 20 kg N/ha).

Atmospheric N deposition

Added daily through `nut_nrain.f90` using ammonium and nitrate concentrations in precipitation.

Plant uptake distribution

Uptake from a layer follows an exponential function of depth controlled by `n_updis` and `p_updis` in `parameters.bsn` (defaults 20). The plant fixes any unmet demand if it is a legume.

In-stream nutrient routing (QUAL2E)

When `bsn_cc%qual2e = 0`, channels use the full QUAL2E water quality formulation in `ch_watqual4.f90`. Simulated state variables are:

- Algae as chlorophyll-a
- Organic N, ammonia, nitrite, nitrate
- Organic P, dissolved P
- CBOD and dissolved oxygen

First-order rates and Arrhenius temperature corrections (theta factors) follow QUAL2E:

```
rate(T) = rate(20) * theta^(T - 20)
```

Default theta values applied in `ch_watqual4.f90`:

Process	Symbol	Theta
Algal growth, respiration	thgra, thrho	1.047
Settling	thrs1-thrs5	1.024 to 1.074
Biological constants	thbc1-thbc4	1.047 to 1.083
Reaeration / decay	thrk1-thrk4	1.024 to 1.060

Key channel nutrient parameters (`channel-nut.cha`):

- `rs1..rs5` settling rates (organic N, organic P, NH4, alg)
- `bc1..bc4` biological rate constants (NH4 to NO2, NO2 to NO3, organic to ammonia hydrolysis, organic P to dissolved P)
- `rk1..rk4` reaeration / BOD decay / SOD
- `ai0, ai1, ai2` stoichiometric ratios for algae
- `lambda0, lambda1, lambda2` light extinction
- `mumax, rhoq` algal growth and respiration

When `bsn_cc%qual2e = 1`, the channel uses simplified transformations.

Switches and parameters

File	Field	Default	Effect
`codes.bsn`	`sol_p_model`	0	0 = original P model, 1 = Vadas and White (2010)
`codes.bsn`	`qual2e`	0	0 = full QUAL2E, 1 = simplified transformations
`parameters.bsn`	`cmn`	0.003	Active organic N mineralisation rate
`parameters.bsn`	`cdn`	1.40	Denitrification exponential rate
`parameters.bsn`	`sdnco`	1.30	Saturation threshold for denitrification (frac of FC)
`parameters.bsn`	`nperco`	0.10	Nitrate percolation/runoff partition
`parameters.bsn`	`nperco_lchtile`	0.50	Nitrate coefficient for tile and bottom-layer leaching
`parameters.bsn`	`phoskd`	175	P soil partitioning coefficient
`parameters.bsn`	`pperco`	10.0	P percolation coefficient
`parameters.bsn`	`psp`	0.40	P availability index
`parameters.bsn`	`rsdco`	0.05	Residue decomposition coefficient
`parameters.bsn`	`decr_min`	0.01	Minimum daily residue decay
`parameters.bsn`	`n_updis`, `p_updis`	20	Plant uptake depth distribution
`parameters.bsn`	`nfixmx`	20	Maximum daily N fixation (kg/ha)

Implementation

Soil N: `nut_nminrl.f90`, `nut_nitvol.f90`, `nut_nlch.f90`, `nut_orgn.f90`, `nut_orgnc.f90`, `nut_orgnc2.f90`, `nut_nrain.f90`, `nut_denit.f90`, `nut_np_flow.f90`.

Soil P: `nut_pminrl.f90`, `nut_pminrl2.f90`, `nut_solp.f90`, `nut_psed.f90`.

Plant uptake and fixation: `pl_nup.f90`, `pl_pup.f90`, `pl_nupd.f90`, `pl_pupd.f90`, `pl_nfix.f90`, `pl_nut_demand.f90`.

Soil pool transfer: `solt_db_read.f90`, `cbn_rsd_decomp.f90`, `cbn_rsd_transfer.f90`, `cbn_surfrsd_decomp.f90`.

In-stream: `ch_watqual4.f90`, `ch_rtday.f90`, `ch_read_nut.f90`, `channel_module.f90`.

Enrichment: `pest_enrsb.f90`.

Related pages

- parameters.bsn
- codes.bsn
- fertilizer.frt
- Carbon
- Plant growth
- Channels

Overview

SWAT+ exposes two soil carbon paths today. The choice is set by the integer `bsn_cc%cswat` in codes.bsn:

cswat	Model	Pools
0	Static (legacy)	Single residue + humus; C is not tracked as a state
1	C-FARM	Reserved for the C-FARM model; not implemented. Treated as off (no carbon model, no `carbon.bsn` required).
2	Active CENTURY/SWAT-C module	Three litter pools and three soil C pools (microbial, slow, passive) following Parton et al. (1993, 1994); driven by `cbn_zhang2.f90`

The carbon model controls how plant residue, manure, tillage, and grazing inputs are routed into the soil and the subsequent CO2, N and P fluxes.

> **Carbon activation value changed to 2**
>
> The dynamic carbon model now activates on `cswat = 2`, matching legacy SWAT numbering (1 = C-FARM, 2 = CENTURY). It was previously selected by `cswat = 1`. `carbon_bsn_read.f90` returns early for any value other than 2, so `cswat = 1` is now inert: no carbon model and no `carbon.bsn` required. This is an input-compatibility break: existing `codes.bsn` files (including refdata and the SWAT+ Editor default) left at `carbon = 1` will silently run with the carbon model off and must be changed to `carbon = 2`.

Process equations

Static (cswat = 0)

Mineralisation and immobilisation are computed in `nut_nminrl.f90`. There is no explicit soil C state. Plant residue carbon is implicit through the `0.42 * biomass` conversion. Residue decomposes at rate

```
decr = max(decr_min, min(1, rsdco_pl * ca * csf))
```

with 80% of mineralised N going to NO3 and 20% to active humus N. See nutrient cycling.

Active carbon module (cswat = 2)

Selected when `bsn_cc%cswat = 2`. The active path runs the full Century pool transformation in `cbn_zhang2.f90`, fed by residue inputs, microbial decomposition, and tillage-driven mixing. Basin-wide and per-layer coefficients come from `carbon.bsn` and `carbon_lyr.bsn`; both files are **required** when `cswat = 2` and are read by `carbon_bsn_read.f90`. The tillage method is set by `bsn_cc%idc_till`:

`idc_till`	Method
1	DSSAT-style
2	EPIC-style
3	Kemanian (default)
4	DNDC-style

In `hru_control.f90`, when `cswat = 2`:

```
call cbn_surfrsd_decomp
call cbn_rsd_transfer
call cbn_zhang2
```

Surface residue decomposes in `cbn_surfrsd_decomp.f90` using the combined temperature-moisture factor `csf` from the N cycle. Below-ground residue is partitioned to soil pools through `cbn_rsd_transfer.f90`. The `cbn_zhang2.f90` routine then runs the Century-style multi-pool turnover that C-FARM and Century share.

The active path also adds the plant biomass increment as carbon (`hpc_d(j)%npp_c`) and accumulates respiratory CO2.

Tillage residue mixing uses `mgt_newtillmix_cswat1.f90` (vs `mgt_newtillmix_cswat0.f90` for the static model). Tillage activity decays over `till_eff_days` days (a column of `carbon.bsn`), controlled per layer by `soil(j)%ly(ly)%tillagef_tillmix`.

Century pool structure (active path)

The Century scheme implemented in `cbn_zhang2.f90`. Pools:

- Litter: structural (`ls`), metabolic (`lm`), with the structural pool further split into lignin (`lsl`) and non-lignin
- Soil organic carbon: microbial biomass (`bm`), slow humus (`hs`), passive humus (`hp`)

For each pool, potential transformation under optimal conditions uses a first-order rate (the `*_para` rates in `carbon_module.f90`). Daily realised transformation is

```
rate(layer) = rate_optimal * cs
cs = sqrt(cdg * sut) * xbm * ox      (cs capped at 10)
```

`cdg` and `sut` are the temperature and water factors from the N cycle. `ox` is the oxygen factor

```
ox = 1 - OX_aa * depth / (depth + exp(OX_aa - OX_bb * depth))
```

with `OX_aa = 10.0`, `OX_bb = 0.035`. `xbm` is the texture and structure factor: 1 in the surface litter and `1 - 0.75 * (silt + clay)` elsewhere.

Default Parton optimal rates (per day):

Pool	Surface	Subsurface
Microbial biomass (BMR)	0.0164	0.02
Slow humus (HSR)	-	0.000548
Passive humus (HPR)	-	0.000012
Metabolic litter (LMR)	0.0405	0.0507
Structural litter (LSR)	0.0107	0.0132

CO2 allocations on transformation:

From	To CO2 (surface)	To CO2 (subsurface)
Biomass	0.60	0.85 - 0.68 * (silt+clay)
Metabolic litter	0.60	0.55
Non-lignin structural	0.60	0.55
Lignin structural	0.30	0.30
Slow humus	0.55	0.55
Passive humus	0.55	0.55

Biomass-to-passive humus allocation in subsurface layers: `0.003 + 0.032 * clay`. Slow-to-passive allocation: `max(0.001, PRMT_45 - 0.00009 * clay)` with `PRMT_45 = 0.003`.

Microbial-biomass leaching uses a water-balance form

```
abl = 1 - exp(-vflow / (0.01 * st + KOC * bd))
```

with `KOC = 4000` (organic carbon partition coefficient, `part_DOC_para`).

Lignin shape factor for structural litter: `xlslf = exp(-3 * lslf)` where `lslf` is lignin fraction of structural litter.

C:N ratios are kept within Parton bounds for each pool and the transformation can release or consume mineral N depending on the deficit (`cpn1..cpn5` and `sum1..sum5`).

The Parton defaults are overridden by the per-layer rates and CO2 fractions supplied in carbon_lyr.bsn; the residue and basin-wide tunables come from carbon.bsn. The legacy `carb_coefs.cbn` and `basins_carbon.tes` files (and their readers `carbon_coef_read.f90` and `carbon_read.f90`) have been removed.

DOC and DIC

The Century implementation accumulates dissolved organic carbon (DOC) and dissolved inorganic carbon (DIC) percolation. Defaults `peroc_DIC_para = 0.95`, `peroc_DOC_para = 0.70`, `hlife_doc_para = 50` days for DOC half-life in groundwater. Source: `carbon_module.f90`, `cbn_zhang2.f90`.

Switches and parameters

File	Field	Default	Effect
codes.bsn	`cswat`	0	0 = static, 1 = C-FARM (reserved, inert), 2 = active CENTURY/SWAT-C module
codes.bsn	`idc_till`	3	Tillage method when `cswat = 2`
carbon.bsn	28 basin-wide scalars	see file	Required when `cswat = 2`; covers initial pool fractions, water coupling, manure partition, biomixing and tillage curves, temperature thresholds, residue C:N / C:P controls, and the slow-humus init method
carbon_lyr.bsn	per-layer rate constants and CO2 fractions	see file	Required when `cswat = 2`; one row per layer group
`carbon_layers.prt`	`cb_n_layers`	max HRU layers	Optional; fixes the number of layers in per-layer carbon output. If absent, defaults to the largest soil layer count across all HRUs

Notable Century coefficients (from `carbon_module.f90`):

Name	Default	Meaning
`CFB_para`	0.42	Carbon fraction of residue
`er_POC_para`	1.5	POC enrichment ratio
`peroc_DOC_para`	0.70	DOC percolation coefficient

Name	Default	Meaning
peroc_DIC_para	0.95	DIC percolation coefficient
part_DOC_para	4000	DOC partition coefficient
hlife_doc_para	50 days	DOC half-life in groundwater
ALSLCO2_para	0.30	Lignin-to-CO2 allocation
APCO2_para	0.55	Passive humus to CO2
ASCO2_para	0.55	Slow humus to CO2
BMR_para_sur / BMR_para_sub	0.0164 / 0.02	Microbial biomass turnover (d^-1)
HPR_para	0.000012	Passive humus turnover (d^-1)
HSR_para	0.000548	Slow humus turnover (d^-1)
PRMT_45_para	0.003	Slow-to-passive allocation coefficient
PRMT_51_para	1.0	Microbial activity coefficient for top layer
OX_aa_para	10.0	Oxygen factor coefficient
OX_bb_para	0.035	Oxygen factor coefficient

Implementation

Core: `cbn_zhang2.f90` (Century pool transformations), `cbn_surfrsd_decomp.f90` (surface residue), `cbn_rsd_decomp.f90` (sub-surface residue, used in legacy paths), `cbn_rsd_transfer.f90` (residue routing into Century pools).

Inputs: `carbon_module.f90` (data types and module-scope tunables), `carbon_bsn_read.f90` (reads both `carbon.bsn` and `carbon_lyr.bsn`), `carbon_layers_read.f90` (optional `carbon_layers.prt`).

Tillage: `mgt_newtillmix_cswat0.f90`, `mgt_newtillmix_cswat1.f90`, `mgt_newtillmix_wet.f90`, `mgt_tillfactor.f90`, `mgt_biomix.f90`.

Driver in `hru_control.f90` switches by `bsn_cc%cswat`. Outputs are written by `output_landscape_init.f90`, `hru_carbon_output.f90`, `lsu_carbon_output.f90`, and `soil_nutcarb_write.f90`. Output gating is per-family through the `hru_cb_*` and `lsu_cb_*` rows of `print.prt`; the old `cbn_diagnostics` flag has been retired.

Related pages

- `codes.bsn`. The `cswat` switch.
- `carbon.bsn`. Basin-wide carbon scalars.
- `carbon_lyr.bsn`. Per-layer carbon coefficients.
- Carbon outputs. HRU and LSU file families.
- Parameter change files. 40 wired carbon calibration parameters.
- Nutrient cycling.
- Plant growth.

Overview

SWAT+ tracks a configurable set of pesticides as constituents. Each pesticide is described by sorption, solubility, half-lives, and (optionally) one or more daughter products. Applications are scheduled by the management table and split between the canopy and the top two soil layers. Once in the system the pesticide decays exponentially on foliage and in the soil, washes off the canopy in rain, leaches with percolation, partitions to runoff water and sediment, and may be taken up by plants. Daughter products are accumulated as the parent decays.

Process equations

Application

Per application, ground cover is

```
gc = (1.99532 - erfc(1.333 * lai_sum - 2)) / 2.1
```

clamped to [0, 1]. Of the applied mass `pest_kg`:

- `gc * pest_kg` goes to plant foliage, partitioned among plants in proportion to their LAI
- `(1 - gc) * surf_frac * pest_kg` goes to soil layer 1
- `(1 - gc) * (1 - surf_frac) * pest_kg` goes to soil layer 2

`surf_frac` is the surface application fraction read from `chem_app.ops`. Source: `pest_apply.f90`.

Decay

First-order decay in soil and on foliage:

```
decay_s = exp(-ln(2) / soil_hlife)
decay_f = exp(-ln(2) / foliar_hlife)
pest(t+1) = pest(t) * decay
```

The decay coefficients `pestcp(k)%decay_s` and `pestcp(k)%decay_f` are pre-computed once from `soil_hlife` and `foliar_hlife` in `pesticide.pst`. Daughter pesticides accumulate decayed mass scaled by the molecular weight ratio:

```
metab_decay = pst_decay * soil_fr * (mol_wt_daughter / mol_wt_parent)
```

Source: `pest_decay.f90`.

Foliar washoff

Each rain day, a fraction `washoff` of the pesticide on each plant's foliage is washed to soil layer 1:

```
pest_soil = washoff * pl_on
```

`washoff` is per pesticide in `pesticide.pst`. Source: `pest_washp.f90`.

Soil partitioning

In each layer, the pesticide is split between adsorbed and dissolved phases using a `Kd` partition computed from soil organic carbon:

```
kd   = koc * cbn / 100
zdb1 = ul + kd * bd * thick
```

`koc` is the organic-carbon-normalised partition coefficient. `zdb1` is the effective partitioning depth in mm. The dissolved concentration available for transport is

```
co = pest * (1 - exp(-vf / zdb1)) / vf
co = min(solub / 100, co)
```

where `vf` is total flow through the layer (`prk + flat`, plus `surfq` in layer 1 and `qtile` in the tile layer). Source: `pest_lch.f90`.

Surface runoff, lateral flow, tile flow, percolation

Surface runoff carries

```
csurf = percop * co
surq  = csurf * surfq
```

Lateral and tile flow use `co` directly (layer 1 lateral uses `csurf`). Percolation moves `co * prk` to the next layer or out of the profile at the bottom layer. `percop` is the basin percolation coefficient (`parameters.bsn`, default 0.5). Source: `pest_lch.f90`.

Sediment-attached transport

For the top layer, sediment-attached pesticide is

```
conc = 100 * kd * pest / (zdb1 + 1e-10)
sed_pest = 0.001 * sedyld * conc * er / area_ha
```

The enrichment ratio `er` is either the channel-specific `erorgn` or the CREAMS-derived value from `pest_enrsb.f90`. Source: `pest_pesty.f90`.

Plant uptake

Daily uptake from each layer is proportional to water uptake by the plant:

```
pest_up = pl_uptake * uptake(ly) / st(ly) * pest(ly)
```

`pl_uptake` is the per-pesticide fraction in `pesticide.pst`. Pesticide on the plant is removed in harvest in proportion to the harvested biomass fraction. Source: `pest_pl_up.f90`, `mgt_harvgrain.f90`.

In-channel and in-aquifer

Channel pesticide reactions use aquatic and benthic half-lives, settling and resuspension velocities, burial, volatilisation, and benthic decay. Source: `ch_rtpest.f90`, `ch_pesticide_module.f90`, `aqu_pesticide_module.f90`, `pest_cha_res_read.f90`.

Switches and parameters

File	Field	Default	Effect
`parameters.bsn`	`percop`	0.5	Pesticide percolation coefficient (0-1). 0 = no pesticide in surface runoff
`pesticide.pst`	`koc`	per pest	Organic-carbon-normalised partition (mL/g)
`pesticide.pst`	`washoff`	per pest	Fraction washed from foliage per rain day
`pesticide.pst`	`foliar_hlife`	per pest	Foliar half-life (days)
`pesticide.pst`	`soil_hlife`	per pest	Soil half-life (days)
`pesticide.pst`	`aq_hlife`, `ben_hlife`	per pest	Aquatic and benthic half-lives (days)
`pesticide.pst`	`solub`	per pest	Water solubility (mg/L)
`pesticide.pst`	`pl_uptake`	per pest	Fraction taken up by plant
`pesticide.pst`	`aq_volat`, `aq_resus`, `aq_settle`, `ben_bury`	per pest	Channel kinetics
`chem_app.ops`	`surf_frac`	per op	Surface application fraction (rest to layer 2)
`pest_hru_aqu.ini`	initial mass	per HRU	Initial pesticide mass per HRU and layer

Implementation

Land phase: `pest_apply.f90`, `pest_decay.f90`, `pest_washp.f90`, `pest_lch.f90`, `pest_pesty.f90`, `pest_pl_up.f90`, `pest_enrsb.f90`, `pest_soil_tot.f90`.

Inputs: `pest_parm_read.f90`, `pest_hru_aqu_read.f90`, `pest_metabolite_read.f90`, `pesticide_data_module.f90`.

Constituent driver: `constituent_mass_module.f90`, `cs_db%num_pests` controls activation.

HRU output: `hru_pesticide_output.f90`.

Channel and aquifer: `ch_rtpest.f90`, `ch_pesticide_module.f90`, `aqu_pesticide_module.f90`, `aqu_pest_output_init.f90`, `aqu_pesticide_output.f90`, `pest_cha_res_read.f90`, `cha_pesticide_output.f90`.

The driver in `hru_control.f90` calls (in order) `pest_apply` (via management), then daily: `pest_washp`, `pest_decay`, `pest_lch`, `pest_pesty`, `pest_pl_up`.

Related pages

- parameters.bsn
- management.sch
- Plant growth
- Channels
- Erosion and sediment

Overview

SWAT+ tracks two optional families of conservative-but-reactive constituents:

- Salt ions (`rtb salt`): SO4, Ca, Mg, Na, K, Cl, CO3, HCO3 (eight ions)
- Trace constituents (`rtb cs`): SeO4, SeO3, boron

Both families share the same flow paths as nitrate: surface runoff, lateral flow, tile drainage, percolation, plant uptake, rainfall and dry deposition, irrigation, and fertiliser/amendment inputs. Salt ions are subject to equilibrium chemistry (precipitation-dissolution of CaCO3, MgCO3, CaSO4, MgSO4, NaCl and cation exchange between Na, Ca, Mg, K). Selenium and boron use simple linear sorption and a first-order reaction term. Salinity stress can reduce crop water uptake.

These modules are active only when their respective databases are present, controlled by `cs_db%num_salts` and `cs_db%num_cs` (set during `cs_read_*` setup).

Process equations

Soil-water transport

For both salts and trace constituents the layer-level transport routine mirrors `nut_nlch.f90`. For each salt ion (or cs species) `m`:

```
vv = perc + flat + (surfq if layer 1) + (qtile if tile layer)
co = mass * (1 - exp(-vv / ul)) / vv
surq_mass  = co * surfq             # layer 1 only
latq_mass  = co * flat
tile_mass  = co * qtile             # at tile layer
perc_mass  = co * prk               # next layer or out of profile
```

Lateral, tile, and percolation are first-order in mobile water with porosity `ul`. No partition coefficient is applied to salt ions; sorbed selenium and boron use `cs_sorb_hru.f90`. Source: `salt_lch.f90`, `cs_lch.f90`.

Salt chemistry in soil

`salt_chem_hru.f90` and `salt_chem_soil_single.f90` solve equilibrium speciation per layer using:

- Activity coefficients via ionic strength (Davies / extended Debye-Huckel form)
- Precipitation-dissolution of five minerals: CaCO3 (Ksp = 3.07e-9), MgCO3 (4.79e-6), CaSO4 (7.89e-5), MgSO4 (7.24e-3), NaCl (37.3)
- Complexation reactions (CaCO3, MgCO3, CaSO4, MgSO4, NaCl)
- Cation exchange between Na, Ca, Mg, K through `cationexchange`

These run iteratively until ion concentrations converge. The same scheme is applied to the aquifer in `salt_chem_aqu.f90` using a separate set of Ksp values (currently the same defaults).

Salinity stress on plants

When `salt_tol_sim = 1` and at least one salt ion is simulated, `pl_waterup.f90` computes a salinity reduction factor per layer using the FAO yield-vs-ECe relationship:

```
soil_TDS_sat = sum(soil_salt_conc) at saturation
soil_ECe = soil_TDS_sat / salt_tds_ec
if (soil_ECe > a):
    reduc_salt = (100 - b * (soil_ECe - a)) / 100
```

a is the salt tolerance threshold (dS/m), b is the slope (yield loss per dS/m above the threshold). Both are per-plant parameters read from salt_plants.slt. If salt_soil_type = 1 (CaSO4 / gypsiferous), the threshold is increased by 2 dS/m. The minimum reduc_salt across rooted layers becomes the plant salt stress factor strss.

salt_effect = 1 applies salinity stress after the other stresses are minimised; salt_effect = 2 (default) includes it in the joint minimum.

Salt uptake by plants

When salt_uptake_on = 1, daily salt removal by plants is read from a user file (salt_uptake.slt) and applied through salt_uptake.f90. Otherwise no plant uptake of salt occurs.

Trace constituent reactions and sorption

cs_rctn_hru.f90 applies first-order reactions to SeO4, SeO3, and boron. Coefficients come from cs_reactions.cs. cs_sorb_hru.f90 runs an equilibrium linear sorption between dissolved and solid-phase pools per layer, parameterised by partition coefficients in cs_data_module.f90. Constituent uptake follows the nitrate-style uptake routine cs_uptake.f90.

Salt and constituent inputs

- Atmospheric: wet deposition (salt_rain.f90, cs_rain.f90) and dry deposition (read from atmodep_salt, atmodep_cs), monthly or annual
- Fertiliser: salt-bearing fertilisers from salt_fert.f90, cs_fert.f90 (read into fert_salt, fert_cs)
- Road salt: salt_roadsalt.f90, applied seasonally per HRU
- Irrigation: salt and cs concentrations in irrigation water (salt_irrig.f90, cs_irrig.f90)
- Initial soil conditions: salt_hru_init.f90, cs_hru_init.f90

In-channel and aquifer

Channels route salt ions through ch_salt_module.f90 and ch_salt_output.f90; constituents through ch_cs_module.f90 and ch_cs_output.f90. Aquifer fluxes are handled in salt_aquifer.f90, salt_chem_aqu.f90, cs_aquifer.f90, cs_rctn_aqu.f90, cs_sorb_aqu.f90. Reservoirs and wetlands have parallel implementations (res_salt.f90, wet_salt.f90).

Switches and parameters

Salts

File	Field	Default	Effect
salt_module.f90	salt_uptake_on	0	1 = enable user-defined daily salt uptake
salt_data_module.f90	salt_tol_sim	0	1 = salinity stress on plant water uptake
salt_data_module.f90	salt_soil_type	0	1 = CaSO4 (gypsiferous), 2 = NaCl
salt_data_module.f90	salt_effect	0	1 = applied after other stresses, 2 = joint minimum
salt_data_module.f90	salt_tds_ec	input	TDS-to-EC conversion factor
salt_data_module.f90	Ksp11..Ksp51	see source	Soil mineral solubility products
salt_data_module.f90	Ksp12..Ksp52	same defaults	Aquifer mineral solubility products
salt_plants.slt	a, b per plant	input	Salt tolerance threshold and slope

File	Field	Default	Effect
salt_recall.slt and other recall files	input	Constituent loads on point sources	

Constituents (cs)

File	Field	Effect
cs_data_module.f90	partition coefficients	Linear sorption per constituent
cs_reactions.cs	first-order rate coefficients	Constituent reaction rates
cs_uptake.cs	uptake fractions per plant	Plant uptake of cs
cs_fert.cs	constituent loads in fertiliser	Inputs from fertiliser
cs_irrig.cs	constituent concentration in irrigation	Inputs from irrigation

Implementation

Salts

Soil and HRU: salt_balance.f90, salt_lch.f90, salt_chem_hru.f90, salt_chem_soil_single.f90, salt_uptake.f90, salt_rain.f90, salt_fert.f90, salt_fert_wet.f90, salt_irrig.f90, salt_roadsalt.f90, salt_hru_init.f90.

Inputs: salt_aqu_read.f90, salt_cha_read.f90, salt_fert_read.f90, salt_hru_read.f90, salt_irr_read.f90, salt_plant_read.f90, salt_roadsalt_read.f90, salt_uptake_read.f90, salt_urban_read.f90.

Modules: salt_module.f90, salt_data_module.f90.

Channel and water bodies: ch_salt_module.f90, ch_salt_output.f90, res_salt.f90, res_salt_module.f90, res_salt_output.f90, res_read_salt_cs.f90, res_read_saltdb.f90, wet_salt.f90, wet_salt_output.f90, wet_read_salt_cs.f90.

Aquifer: salt_aquifer.f90, salt_chem_aqu.f90, aqu_salt_output.f90.

Climate, recall, export coefficients, delivery ratios: cli_read_atmodep_salt.f90, recall_read_salt.f90, recall_salt.f90, dr_read_salt.f90, exco_read_salt.f90, header_salt.f90.

Output: hru_salt_output.f90, ru_salt_output.f90, output_ls_salt_module.f90.

Trace constituents (cs)

Soil and HRU: cs_balance.f90, cs_lch.f90, cs_rctn_hru.f90, cs_sorb_hru.f90, cs_uptake.f90, cs_rain.f90, cs_fert.f90, cs_fert_wet.f90, cs_irrig.f90, cs_hru_init.f90.

Inputs: cs_aqu_read.f90, cs_cha_read.f90, cs_fert_read.f90, cs_hru_read.f90, cs_irr_read.f90, cs_plant_read.f90, cs_reactions_read.f90, cs_uptake_read.f90, cs_urban_read.f90.

Modules: cs_module.f90, cs_data_module.f90.

Channel: ch_cs_module.f90, ch_cs_output.f90, cs_str_output.f90.

Aquifer: cs_aquifer.f90, cs_rctn_aqu.f90, cs_sorb_aqu.f90, aqu_cs_output.f90.

Reservoirs and wetlands: res_cs_module.f90, res_cs_output.f90, wet_cs_output.f90.

HRU and RU output: hru_cs_output.f90, ru_cs_output.f90.

Related pages

- Nutrient cycling
- Plant growth
- Aquifers

- Channels
- Recall and point sources

Overview

Channels move water, sediment, and constituents along the drainage network from upstream HRUs and aquifers to the outlet. Each channel object has a geometry (width, depth, slope, length, side slope, Manning's n) read from `hydrology-cha.cha` and `sd_channel.cha`, and a rating curve built from that geometry. Daily inflow is the sum of all upstream object outflows. Daily outflow is computed by one of two routing methods selected by `bsn_cc%rte`. After routing, the channel runs sediment transport, in-stream water quality, and optional pesticide, salt, and constituent routines.

Process equations

Channel routing: variable storage vs Muskingum

`bsn_cc%rte` in `codes.bsn` selects the method:

`bsn_cc%rte`	Method
0	Variable storage coefficient
1	Muskingum

Both run inside `ch_rtmusk.f90` (the file name reflects history; the routine dispatches on `bsn_cc%rte` inside its sub-step loop). The older daily-only path in `ch_rtday.f90` uses the variable storage coefficient and is retained for the legacy channel object.

Variable storage. At each sub-step the outflow fraction is

```
scoef = bsn_prm%scoef * 2 * dt / (2 * ttime + dt)
outflow = scoef * total_storage
```

where `ttime` is the travel time interpolated from the channel rating curve at the current flow rate. `bsn_prm%scoef` (default 1.0) calibrates the storage coefficient.

Muskingum. The three-coefficient form:

```
O2 = c1 * I2 + c2 * I1 + c3 * O1
```

with `c1`, `c2`, `c3` derived from the storage time constant `Km` and the weighting factor `X`. `Km` is computed from the bankfull and low-flow reach response times scaled by `bsn_prm%msk_co1` (default 0.75) and `bsn_prm%msk_co2` (default 0.25). `X = bsn_prm%msk_x` (default 0.20). The number of sub-daily steps and stability sub-steps is `sd_ch(jrch)%msk%nsteps` and `sd_ch(jrch)%msk%substeps`.

After routing, transmission losses are subtracted using the channel bed conductivity `sd_ch%chk` (mm/h) applied over the wetted perimeter and reach length. Reach evaporation uses `bsn_prm%evrch` (default 0.60) applied to PET over the channel surface area.

Sediment transport: legacy vs sd_channel

Two sediment routines exist.

Legacy stream power transport (`channel_module.f90`). Older channels carry the maximum sediment concentration as a power function of the peak flow velocity:

```
conc_max = spcon * v_peak ^ spexp
```

`spcon` and `spexp` live in `bsn_prm` as legacy parameters; in current `basin_module.f90` they are flagged as not used by the modern routine. TODO: verify whether the legacy channel path still references these.

Velocity and shear (`sd_channel_sediment3.f90`). The current `sd_channel` path computes:

1. Peak channel flow rate from the daily flow and a peak ratio `sd_ch%pk_rto` adjusted for drainage area.
2. Channel velocity from Manning's equation at the peak rate, using the rating curve to get cross-section area and hydraulic radius.
3. Critical bank velocity `vel_cr` from the bank cohesion and bulk density factors, combined with the side-slope geometry.
4. Bank erosion as a function of the ratio `vel / vel_cr` raised to `sd_chd1%bed_exp` (default 1.5), distributed along the meander arc length scaled by `sd_chd1%arc_len_fr` (default 1.2).
5. Bed erosion from a critical shear stress comparison using the median bed particle diameter `sd_chd%d50` and the bulk density `sd_chd%ch_bd`.
6. Deposition computed in the flood plain when the flow exceeds bankfull, with a trap efficiency `trap_eff` set by the flood plain Manning's n `sd_chd%fpn` and slope `sd_chd%fps`.

Eroded sediment is split between channel and bank; a fraction `sd_chd1%wash_bed_fr` (default 0.1) of bank erosion is washload that leaves the reach. Particle-class fractions (silt, clay, sand, large aggregates, gravel) follow the bed composition.

In-stream water quality (QUAL2E)

Nutrient transformations are computed in `ch_watqual4.f90`. The implementation is the QUAL2E equation set: algal growth limited by light, nutrient availability, and temperature; oxygen balance between algal photosynthesis, respiration, BOD decay, nitrification, and reaeration; nitrogen cycling from organic N to ammonium to nitrite to nitrate; phosphorus cycling from organic P to mineral P with settling and benthic sources.

Temperature corrections use the standard theta factors hard-coded in the routine (`thgra = 1.047`, `thrho = 1.047`, `thbc1 = 1.083`, etc.). Reaeration uses `bc1`, `bc2`, `bc3`, `bc4`, `rs1` through `rs5`, and `rk1` through `rk4` read from `nutrients-cha.nut`.

When `bsn_cc%qual2e = 1`, a simplified soluble-to-particulate transformation is applied after `ch_watqual4` runs, overriding the QUAL2E organic and mineral exchange:

```
delta_orgn = (1 - exp(-n_sol_part * ttime)) * orgn_in
delta_orgp = (1 - exp(-p_sol_part * ttime)) * sedp_in
```

`n_sol_part` and `p_sol_part` come from `sd_channel.cha` (defaults 0.01 each). `ttime` is the channel travel time from the rating curve.

Pesticides, salts, constituents

After water quality, the channel runs `ch_rtpest.f90` for pesticides and the salt and constituent routines if those modules are active. See Pesticides and Salts and constituents.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%rte`	0	`codes.bsn`	0 = variable storage; 1 = Muskingum
`bsn_cc%qual2e`	0	`codes.bsn`	1 = simplified N and P sol-to-part overrides QUAL2E
`bsn_prm%scoef`	1.0	`parameters.bsn`	Variable-storage channel coefficient
`bsn_prm%msk_co1`	0.75	`parameters.bsn`	Muskingum bankfull storage coefficient
`bsn_prm%msk_co2`	0.25	`parameters.bsn`	Muskingum low-flow storage coefficient
`bsn_prm%msk_x`	0.20	`parameters.bsn`	Muskingum weighting factor
`bsn_prm%evrch`	0.60	`parameters.bsn`	Reach evaporation factor

Switch / parameter	Default	File	Effect
bsn_prm%spcon	0.0	parameters.bsn	Legacy stream power coefficient (not used by sd_channel)
bsn_prm%spexp	0.0	parameters.bsn	Legacy stream power exponent (not used by sd_channel)
bsn_prm%ch_d50	0.0 mm	parameters.bsn	Basin-wide channel median particle size
sd_chd%chw	per channel	sd-channel	Channel width (m)
sd_chd%chd	per channel	sd-channel	Channel depth (m)
sd_chd%chs	per channel	sd-channel	Channel slope (m/m)
sd_chd%chl	per channel	sd-channel	Channel length (km)
sd_chd%chn	per channel	sd-channel	Channel Manning's n
sd_chd%chk	per channel	sd-channel	Channel bottom conductivity (mm/h)
sd_chd%cov	per channel	sd-channel	Channel cover factor (0-1)
sd_chd%d50	per channel	sd-channel	Channel median sediment size (mm)
sd_chd%ch_bd	per channel	sd-channel	Bed and bank bulk density (t/m^3)
sd_chd%ch_clay	per channel	sd-channel	Bed and bank clay percent
sd_chd1%pk_rto	1.0	sd-channel	Ratio of peak to mean daily flow
sd_chd1%n_setl	0.5	sd-channel	N settling ratio to sediment settling
sd_chd1%p_setl	0.5	sd-channel	P settling ratio to sediment settling
sd_chd1%n_sol_part	0.01	sd-channel	Simplified soluble to particulate N rate
sd_chd1%p_sol_part	0.01	sd-channel	Simplified soluble to particulate P rate
sd_chd1%bed_exp	1.5	sd-channel	Bed erosion exponential coefficient
sd_chd1%arc_len_fr	1.2	sd-channel	Fraction of arc length where bank erodes
sd_chd1%wash_bed_fr	0.1	sd-channel	Washload fraction of bank erosion

Implementation

Source modules in `swatplus/src/`:

- `sd_channel_control3.f90` is the master control routine for an `sd_channel` object. It receives the daily inflow, calls routing, sediment, water quality, pesticide, salt, and constituent routines, and updates output structures.
- `ch_rtmusk.f90` runs the sub-daily routing loop. It dispatches on `bsn_cc%rte` and computes outflow, transmission loss, and reach evaporation.
- `ch_rtday.f90` is the daily variable-storage routing used by the legacy `channel` object.
- `sd_channel_sediment3.f90` computes channel velocity, critical velocity, bank and bed erosion, and flood plain deposition for the `sd_channel` path.
- `ch_watqual4.f90` performs the QUAL2E in-stream nutrient and oxygen calculations.
- `ch_rtpest.f90` routes pesticides through the channel.
- `ch_temp.f90` computes channel water temperature.
- `channel_module.f90` defines the legacy `channel` derived types and storage arrays.
- `sd_channel_module.f90` defines the `sd_channel` derived types: hydsed data (`sd_chd`), sediment-nutrient data (`sd_chd1`), sediment budget output, and morphology output.
- `channel_data_module.f90` holds channel parameter arrays read from input.
- `channel_velocity_module.f90` stores the rating curve and velocity bookkeeping.

- `ch_rchinit.f90` initialises the channel state at the start of simulation.
- `ch_ttcoef.f90` builds the Muskingum coefficients from the channel geometry.

Related pages

- `codes.bsn` for `rte` and `qual2e`.
- `parameters.bsn` for Muskingum and storage coefficients.
- `sd-channel` for channel geometry and sediment parameters.
- `cha` for the legacy channel object.
- Channel sediment and nutrient output.
- Hydrology for the upstream HRU inflow.
- Aquifers for return flow to channels.

Overview

The aquifer cluster receives percolation from HRU soil profiles and returns it to channels as baseflow, sends a fraction to deep seepage, and supports root uptake (revap) when the water table is shallow. Two implementations are available. The default is a 1D lumped recession aquifer with one storage per aquifer object. The alternative is the `gwflow` module, a grid-cell groundwater model with explicit head and lateral exchange. `bsn_cc%gwflow` in `codes.bsn` switches between them.

Process equations

1D recession aquifer (`bsn_cc%gwflow = 0`)

The default aquifer object holds a single storage. Each day the recharge from the soil profile is added to storage, then the water table depth, return flow, deep seepage, and revap are computed in `aqu_1d_control.f90`.

Water table depth. Storage converts to a water table depth using specific yield:

```
dep_wt = dep_bot - stor / (1000 * spyld)
```

`dep_bot` is the aquifer bottom depth (m), `spyld` is specific yield ($m^{3/m}3$), and `stor` is storage in mm.

Return flow. When the water table is at or above the threshold `flo_min`, return flow follows a recession with lag factor `alpha`:

```
flo_new = flo_prev * exp(-alpha) + rchrg * (1 - exp(-alpha))
```

`alpha_e = exp(-alpha)` is precomputed once. Return flow is capped at the available storage. Below the threshold (`dep_wt > flo_min`) return flow is zero.

Deep seepage. A fixed fraction of the daily recharge seeps to the deep aquifer:

```
seep = rchrg * seep_frac
```

where `seep_frac` is `aqu_dat%seep` from `aquifer.aqu`.

Revap. When the water table is shallower than `revap_min`, plant uptake and evaporation from the aquifer is

```
revap = pet * revap_co
```

capped at storage. Above the threshold revap is zero. `revap_co` and `revap_min` are read from `aquifer.aqu`.

Nitrate. Nitrate in recharge water mixes with the aquifer pool. Return flow nitrate is the storage-weighted concentration. A first-order decay uses the half-life `hlife_n` (default 30 days) read from `aquifer.aqu`. Nitrate seepage uses the same concentration applied to the seepage volume.

2D gwflow (bsn_cc%gwflow = 1)

When bsn_cc%gwflow = 1 the lumped 1D aquifer is replaced by a cell-based groundwater model. The watershed is discretised into a grid (structured or unstructured) where each active cell carries a head, hydraulic conductivity, specific yield, and a list of connections to neighbour cells. The model couples to channels, reservoirs, wetlands, tile drains, pumping wells, and HRU recharge.

Per-cell state (in gwflow_module.f90):

- head current simulated head (m)
- hydc aquifer hydraulic conductivity (m/day)
- spyd specific yield ($m^{3/m}3$)
- exdp groundwater ET extinction depth (m)
- thck aquifer thickness, botm bedrock elevation, elev surface elevation
- stor available groundwater storage (m^3)

Each day the model:

1. Computes recharge from HRU or LSU connections (gwflow_rech.f90).
2. Computes groundwater ET and phreatophyte transpiration (gwflow_gwet.f90, gwflow_phreatophyte.f90).
3. Exchanges flow with channels using the Darcian conductance derived from the channel bed (gwflow_channel_exch.f90).
4. Handles flow to and from reservoirs and ponds (gwflow_reservoir.f90, gwflow_pond.f90).
5. Handles wetland exchange (gwflow_wetland.f90).
6. Handles tile drain capture (gwflow_tile.f90).
7. Handles floodplain exchange when channels overflow (gwflow_floodplain.f90).
8. Handles canal diversions and external inflows (gwflow_canal.f90, gwflow_canal_div.f90, gwflow_canal_ext.f90).
9. Handles pumping (gwflow_pump_allo.f90, gwflow_pump_ext.f90).
10. Solves the head at each cell from neighbour conductances and storage change (gwflow_simulate.f90).
11. Redistributes saturation-excess water if a cell head exceeds the surface (gwflow_satexcess.f90).
12. Routes solutes and heat through cells (gwflow_solute.f90, gwflow_chem.f90, gwflow_heat.f90).

Aquifer-to-channel mapping for the geomorphic baseflow option uses aqu2d_init.f90 to sort channels by drainage area, allocate flow to channels by length and area, and aqu2d_read.f90 to read the aquifer_cha.con link file.

Boundary conditions are set per cell (gw_state(i)%stat): 0 inactive, 1 active, 2 boundary. Constant-head and no-flow types are selected by bc_type in the gwflow inputs.

Switches and parameters

Switch / parameter	Default	File	Effect
bsn_cc%gwflow	0	codes.bsn	0 = 1D recession aquifer; 1 = gwflow grid model
aqu_dat%flo	0.05 mm	aqu	Initial flow from aquifer
aqu_dat%dep_bot	per aquifer	aqu	Depth to aquifer bottom (m)
aqu_dat%dep_wt	per aquifer	aqu	Initial water table depth (m)
aqu_dat%bf_max	0.0 mm	aqu	Maximum daily baseflow
aqu_dat%alpha	per aquifer	aqu	Recession lag factor (1/days)
aqu_dat%revap_co	per aquifer	aqu	Revap coefficient
aqu_dat%revap_min	per aquifer	aqu	Water table depth threshold for revap (m)
aqu_dat%flo_min	per aquifer	aqu	Water table depth threshold for return flow (m)
aqu_dat%seep	per aquifer	aqu	Fraction of recharge to deep seepage
aqu_dat%spyld	per aquifer	aqu	Specific yield ($m^{3/m}3$)

Switch / parameter	Default	File	Effect
aqu_dat%hlife_n	30 days	aqu	Half-life of nitrate in groundwater
aqu_dat%no3	per aquifer	aqu	Initial NO3-N concentration (ppm)
aqu_dat%minp	per aquifer	aqu	Initial mineral P concentration (ppm)
gw_state%hydc	per cell	gwflow inputs	Hydraulic conductivity (m/day)
gw_state%spyd	per cell	gwflow inputs	Specific yield ($m^3/m3$)
gw_state%exdp	per cell	gwflow inputs	ET extinction depth (m)
conn_type	0	gwflow inputs	1 = HRU recharge connections; 2 = LSU
bc_type	0	gwflow inputs	1 = constant head; 2 = no-flow

TODO: verify exact column order and defaults in `aquifer.aqu` and the gwflow input files against the corresponding `*_read.f90` readers.

Implementation

Source modules in `swatplus/src/`:

- `aqu_1d_control.f90` is the master daily driver for the 1D aquifer object. Adds recharge, computes water table, return flow, deep seepage, revap, and nitrate transport.
- `aqu_read.f90` reads `aquifer.aqu` into the `aqudb` and `aqu_dat` arrays.
- `aqu_read_init.f90`, `aqu_read_init_cs.f90` read initial conditions.
- `aqu_initial.f90` and `aqu_initial` set up storage and pre-compute `alpha_e`.
- `aquifer_module.f90` defines `aquifer_database`, `aquifer_data_parameters`, and `aquifer_dynamic` derived types.
- `aquifer_output.f90`, `aqu_pesticide_output.f90`, `aqu_salt_output.f90`, `aqu_cs_output.f90` print aquifer state and constituent outputs.
- `aqu2d_init.f90` orders channels by drainage area and sets up the aquifer-to-channel mapping for the geomorphic baseflow option.
- `aqu2d_read.f90` reads the `aquifer_cha.con` link file.
- `gwflow_module.f90` defines all gwflow state types and arrays.
- `gwflow_read.f90` reads the gwflow grid and parameters.
- `gwflow_simulate.f90` is the daily driver: assembles fluxes and solves for new heads.
- `gwflow_rech.f90` distributes HRU or LSU recharge to cells.
- `gwflow_channel_exch.f90`, `gwflow_chan_read.f90` handle cell-channel exchange.
- `gwflow_floodplain.f90` handles overland exchange with the channel flood plain.
- `gwflow_reservoir.f90`, `gwflow_wetland.f90`, `gwflow_pond.f90` handle waterbody exchange.
- `gwflow_tile.f90` handles tile drainage capture.
- `gwflow_canal.f90`, `gwflow_canal_div.f90`, `gwflow_canal_ext.f90` handle canal networks.
- `gwflow_pump_allo.f90`, `gwflow_pump_ext.f90` handle pumping.
- `gwflow_lateral.f90` handles lateral inflow from non-grid sources.
- `gwflow_soil.f90` couples grid cells back into HRU soil profiles.
- `gwflow_satexcess.f90` handles saturation-excess water when head reaches the surface.
- `gwflow_gwet.f90`, `gwflow_phreatophyte.f90` handle groundwater ET.
- `gwflow_solute.f90`, `gwflow_chem.f90` handle nitrate, salt, and constituent transport.
- `gwflow_heat.f90` handles heat transport.
- `gwflow_output.f90` writes daily, monthly, yearly, and average-annual gwflow output.

Related pages

- codes.bsn for the `gwflow` switch.

- aqu for the 1D aquifer input file.
- Hydrology for percolation that becomes aquifer recharge.
- Channels for baseflow return to streams.
- Aquifer, reservoir, and plant output.

Overview

Reservoirs are standalone water body objects in the routing network. They receive inflow from upstream channels, HRUs, or recall objects, store it, and release it according to a decision table in `res_rel.dtl`. Each reservoir runs sediment settling, in-pond nutrient processes, and optional pesticide, salt, and constituent processes.

Wetlands are a separate object type linked to individual HRUs. They sit on the HRU surface, intercept surface runoff before it reaches the channel, and run a similar water balance and sediment-nutrient set as reservoirs. Wetland storage is configured per HRU in `wetland.wet`.

Process equations

Reservoir water balance

The daily water balance for a reservoir in `res_control.f90` is

```
stor_new = stor_prev + inflow + precip - outflow - evap - seep - irrig
```

where:

- `inflow` is the upstream hydrograph (`ht1`).
- `precip` is precipitation onto the water surface from the assigned weather station, using the current surface area.
- `outflow` is the release computed by `res_hydro.f90`.
- `evap` is potential evaporation times the water surface area, optionally adjusted by the reservoir evaporation coefficient.
- `seep` is seepage through the bed, computed from the bed hydraulic conductivity over the surface area.
- `irrig` is water withdrawn for irrigation through the water allocation module.

Surface area as a function of storage is interpolated from the principal volume `pvol_m3` and emergency volume `evol_m3` defined for the reservoir.

Release rules via `res_rel.dtl`

Releases are not hard-coded. Each reservoir points to a decision table in `res_rel.dtl`. The table lists conditions (volume above principal, day of year, downstream demand, …) and corresponding actions (weir release, target release, percent of volume, irrigation supply, …). At each daily step `res_hydro.f90`:

1. Walks the action list (`d_tbl%acts`).
2. For each action, evaluates whether all condition alternatives are satisfied (`d_tbl%act_hit`).
3. Fires the corresponding release equation.

Release types include:

- `weir_release` for a weir of height `wbody_prm%weir_hgt` and width `wbody_prm%weir_wid`. The discharge over time follows the weir equation with the daily volume above the weir crest as the head source.
- `pvol_release` and `evol_release` for releases triggered when storage exceeds the principal or emergency volume.
- Doell-style storage-based release with coefficient `kr` and exponent `alpha` (Jose T 2025 extension in `res_hydro.f90`).
- Hanazaki monthly-target release using monthly inflow and demand.
- HYPE seasonal sine and linear storage modulation for hydropower-style operations.

The weir branch calls `res_weir_release.f90` to integrate over sub-daily steps.

Reservoir sediment

`res_sediment.f90` computes settling using a first-order rate. If the in-reservoir sediment concentration (ppm) exceeds the equilibrium concentration `wbody_prm%sed%nsed`, the excess is reduced by the settling coefficient `wbody_prm%sed_stlr_co`:

```
sed_ppm_new = (sed_ppm - nsed) * sed_stlr_co + nsed
```

After settling, sand, large aggregates, silt, clay, and gravel are recomputed; all of the sand-sized fraction is assumed to deposit. Outflow sediment leaves at the post-settling concentration.

Reservoir nutrients

`res_nutrient.f90` runs an in-pond nutrient set: settling of organic N and organic P, mineralisation, nitrification, and algal growth limited by light and nutrients. Soluble nutrients track storage-weighted concentration and leave at the outflow concentration. Settling rates are read from `nutrients-res.nut`.

Reservoir pesticides, salts, constituents

`res_pest.f90`, `res_salt.f90`, and `res_cs.f90` run when those constituent modules are active. See Pesticides and Salts and constituents.

Wetlands on HRUs

Wetlands are configured per HRU via `wet_initial.f90` and `wetland.wet`. Each wetland has:

- Principal volume `wet_ob%pvol` (m^3) computed from `wet_hyd%pdep` (principal depth, mm) times the HRU area.
- Emergency volume `wet_ob%evol` (m^3) from `wet_hyd%edep`.
- Principal and emergency surface areas `psa` and `esa`.
- Optional weir geometry `weir_hgt` and `weir_wid` read from `weir.res`.

The wetland state `wet(iihru)` is the same `water_body` type used for reservoirs. In `hru_control.f90` the HRU surface runoff is routed into the wetland (`wet(j) = wet(j) + ht1`), the wetland runs its water balance and release (using the same `res_hydro` machinery on a wetland), and the outflow is added to the HRU outgoing hydrograph. Storage to depth uses a quadratic relation from `wet_hyd%bcoef` and `wet_hyd%ccoef`:

```
depth^2 * ccoef + depth * bcoef = (1 - flo / pvol)
```

with the positive root taken.

Wetlands run their own sediment, nutrient, pesticide, salt, and constituent routines using the same module code paths as reservoirs.

Switches and parameters

Switch / parameter	Default	File	Effect
`res_hyd%iyres`	per reservoir	`hydrology-res.res`	Year reservoir comes online
`res_hyd%mores`	per reservoir	`hydrology-res.res`	Month reservoir comes online
`pvol_m3`	per reservoir	`hydrology-res.res`	Principal volume (m^3)
`evol_m3`	per reservoir	`hydrology-res.res`	Emergency volume (m^3)
`wbody_prm%sed%nsed`	per reservoir	`sediment-res.sed`	Equilibrium sediment concentration (ppm)
`wbody_prm%sed_stlr_co`	per reservoir	`sediment-res.sed`	Sediment settling coefficient
`wbody_prm%weir_hgt`	per reservoir	`weir.res`	Weir height (m)
`wbody_prm%weir_wid`	per reservoir	`weir.res`	Weir width (m)

Switch / parameter	Default	File	Effect
`res_ob%release_dtl`	per reservoir	`dtl` (`res_rel.dtl`)	Decision table name for release rules
`wet_hyd%pdep`	per HRU wetland	`wetland.wet`	Principal depth (mm)
`wet_hyd%edep`	per HRU wetland	`wetland.wet`	Emergency depth (mm)
`wet_hyd%psa`	per HRU wetland	`wetland.wet`	Principal surface area fraction of HRU
`wet_hyd%esa`	per HRU wetland	`wetland.wet`	Emergency surface area fraction of HRU
`wet_hyd%bcoef`	per HRU wetland	`wetland.wet`	Storage-depth quadratic coefficient
`wet_hyd%ccoef`	per HRU wetland	`wetland.wet`	Storage-depth quadratic coefficient

TODO: verify default values in `hydrology-res.res`, `sediment-res.sed`, `nutrients-res.nut`, `weir.res`, and `wetland.wet` against their respective `_read.f90` readers.

Implementation

Source modules in `swatplus/src/`:

- `res_control.f90` is the master daily driver for a reservoir object. Adds inflow, computes the water balance, calls release, sediment, nutrient, pesticide, salt, and constituent routines.
- `res_hydro.f90` evaluates the release decision table and computes outflow. Houses weir, principal-volume, emergency-volume, Doell, Hanazaki, and HYPE release branches.
- `res_weir_release.f90` integrates weir discharge over sub-daily steps.
- `res_rel_conds.f90` evaluates release conditions specific to reservoirs.
- `res_sediment.f90` computes in-reservoir sediment settling.
- `res_nutrient.f90` runs in-reservoir nutrient transformations.
- `res_pest.f90` handles reservoir pesticides.
- `res_salt.f90`, `res_cs.f90` handle salts and constituents.
- `res_initial.f90`, `res_allo.f90`, `res_objects.f90` set up reservoir state at simulation start.
- `res_read.f90`, `res_read_hyd.f90`, `res_read_sed.f90`, `res_read_nut.f90`, `res_read_weir.f90`, `res_read_init.f90`, `res_read_conds.f90`, `res_read_elements.f90`, `res_read_csdb.f90`, `res_read_saltdb.f90`, `res_read_salt_cs.f90` read the reservoir input files.
- `reservoir_module.f90` defines reservoir types including `wet_ob` for wetlands.
- `reservoir_data_module.f90` and `reservoir_conditions_module.f90` hold parameter and conditions arrays.
- `reservoir_output.f90`, `res_pesticide_output.f90`, `res_salt_output.f90`, `res_cs_output.f90` write reservoir outputs.
- `wet_initial.f90` initialises a wetland on each flagged HRU.
- `wet_all_initial.f90` initialises all wetlands at simulation start.
- `wet_fp_init.f90` sets up the wetland-floodplain link to channels.
- `wet_read.f90`, `wet_read_hyd.f90`, `wet_read_salt_cs.f90` read wetland input files.
- `wet_irrp.f90` handles irrigation withdrawal from a wetland.
- `wet_cs.f90`, `wet_cs_output.f90`, `wet_salt.f90`, `wet_salt_output.f90` handle wetland constituents and salts.

Related pages

- Decision tables for how release rules are evaluated.
- `dtl` for the decision table input file format.
- `res` for the reservoir input files.
- `wet` for the wetland input files.
- Channels for downstream routing.
- Aquifer, reservoir, and plant output.

Overview

Three related object types let SWAT+ inject water and constituent loads into the routing network from outside the simulated land surface: recall objects, export coefficient objects, and delivery ratio objects.

- **Recall** objects deliver a time series of flow and mass (NO3, NH3, organic N, mineral P, organic P, sediment, pesticides, salts, constituents) read from `recall.rec` and supporting per-object data files. They are the standard point source representation, including municipal and industrial discharges, atmospheric deposition points, or measured imports.
- **Export coefficient** objects deliver an average annual flow and mass per unit area, computed once and applied each day. They are useful where only annual loading rates are known.
- **Delivery ratio** objects scale incoming loads on a routing unit basis (delivery from HRUs to channels), either calculated from time of concentration or set to a fixed fraction.

All three share a common reader infrastructure under `recall_module.f90` because internally they are all stored as recall databases.

Process equations

Recall objects

The recall reader (`recall_read.f90`) opens the file pointed to in `recall.rec` for each recall object and selects an allocation pattern based on the time step token:

`tstep`	Storage shape	Meaning
`sub`	`hyd_flo(step*366, nbyr)` plus `hd(366, nbyr)`	Sub-daily values per simulation year
`day`	`hd(366, nbyr)`	One record per day
`mo`	`hd(12, nbyr)`	One record per month
`yr`	`hd(1, nbyr)`	One record per year

At runtime the recall driver looks up the current day, month, or year and copies the stored hydrograph (`recall(irec)%hd(...)`) into the object's outgoing hydrograph. For sub-daily, flow is stored as `m^3/s` and converted to volume per step (`flo * 86400 / time%step`).

Recall objects can be linked to channel objects so the volume is added to the channel inflow, or to other receiving objects through the connectivity file `recall.con`.

Negative-flow recall objects represent diversions: `recall_nut.f90` and `recall_salt.f90` remove constituent mass from the source channel in proportion to the diverted volume and the source-water concentration.

Export coefficient objects

Export coefficient objects (`exco_module.f90`) are stored in the same recall database type with one record per object that holds average annual flow and constituent loads. The reader chain (`exco_db_read.f90`, `exco_read_om.f90`, `exco_read_pest.f90`, `exco_read_path.f90`, `exco_read_hmet.f90`, `exco_read_salt.f90`) populates separate organic-mineral, pesticide, pathogen, heavy metal, and salt files keyed by `exco_*_name`. Each day the export coefficient flow is divided by 365.25 (TODO: verify exact daily fraction) and added to the receiving object.

Delivery ratio objects

Delivery ratios scale incoming loads from a routing-unit element by a fraction between 0 and 1. The element database `dr_db` in `dr_module.f90` lists the organic-mineral, pesticide, pathogen, heavy metal, and salt delivery files for each delivery ratio object.

In `dr_ru.f90` each element of a routing unit gets a `dr` value built from the `dr_name` keyword:

```
if (ru_elem(ielem)%dr_name == "calc" .or. "0") then
   ! compute from time of concentration
```

```
    rto = tconc(ihru) / ru_tc(iru)
    rto = min(1.0, sqrt(rto))
    ru_elem(ielem)%dr = rto + hz
    ru_elem(ielem)%dr%flo = 1.
else if (ru_elem(ielem)%dr_name == "full" .or. "1") then
    ru_elem(ielem)%dr = 1. + hz
end if
```

Three cases:

- `"calc"` or `"0"`: ratio is the square root of the HRU time of concentration over the routing unit time of concentration, capped at 1. Flow itself is delivered in full (`dr%flo = 1`).
- `"full"` or `"1"`: full delivery, ratio is 1 for everything.
- A named `dr_db` entry: ratios are read from the corresponding `_om`, `_pest`, `_path`, `_hmet`, and `_salt` files.

Delivery ratios apply only to the mass components (sediment, N, P, pesticide, salt), not to flow.

Switches and parameters

Parameter	File	Effect
`recall_db%org_min%name`	`recall.rec`	Organic and mineral data file for this recall object
`recall_db%org_min%units`	`recall.rec`	`mass` or `conc`
`recall_db%org_min%tstep`	`recall.rec`	`sub`, `day`, `mo`, or `yr`
`recall_db%pest%name`	`recall.rec`	Pesticide data file
`recall_db%path%name`	`recall.rec`	Pathogen data file
`recall_db%hmet%name`	`recall.rec`	Heavy metal data file
`recall_db%salt%name`	`recall.rec`	Salt data file
`recall_db%constit%name`	`recall.rec`	Generic constituent data file
`exco_db%om_file`	`exco.exc`	Organic-mineral export coefficient file
`exco_db%pest_file`	`exco.exc`	Pesticide export coefficient file
`exco_db%salts_file`	`exco.exc`	Salt export coefficient file
`dr_db%om_file`	`delratio.del`	Organic-mineral delivery ratio file
`dr_db%pest_file`	`delratio.del`	Pesticide delivery ratio file
`dr_db%salts_file`	`delratio.del`	Salt delivery ratio file
`ru_elem%dr_name`	`rout_unit.def`	Delivery ratio choice: `calc`, `full`, or a named `dr_db` entry

TODO: verify exact file names and column layouts against the source readers and current refdata.

Implementation

Source modules in `swatplus/src/`:

- `recall_module.f90` defines `constituent_file_data` and `recall_databases`. The same `recall_db` type is used for recall, exco, and dr objects.
- `recall_read.f90` reads `recall_db.rec` (the database index) and dispatches per-object reads. For each recall it opens the organic-mineral file, allocates the storage for the selected time step, and stores hydrographs by day, month, or year.
- `recall_read_salt.f90` reads the salt data file.
- `recall_read_cs.f90` reads the generic constituent data file.
- `recall_nut.f90` removes nutrient mass from a source channel for negative-flow recall objects (diversions).
- `recall_salt.f90` does the same for salts.
- `recall_cs.f90` does the same for constituents.
- `recall_output.f90` writes recall object output.
- `exco_module.f90` defines `export_coefficient_datafiles` and the per-constituent name and number arrays.
- `exco_db_read.f90` reads `exco.exc`.

- `exco_read_om.f90`, `exco_read_pest.f90`, `exco_read_path.f90`, `exco_read_hmet.f90`, `exco_read_salt.f90` read the per-constituent export coefficient files.
- `dr_module.f90` defines `delivery_ratio_datafiles` and the per-constituent name and number arrays.
- `dr_db_read.f90` reads `delratio.del`.
- `dr_read_om.f90`, `dr_read_pest.f90`, `dr_path_read.f90`, `dr_read_hmet.f90`, `dr_read_salt.f90` read the per-constituent delivery ratio files.
- `dr_ru.f90` computes the per-element delivery ratio for `calc` and `full` cases and assigns named delivery ratios at runtime.

Related pages

- Channels for the receiving object.
- Decision tables for management triggers that may use recall objects.
- Nutrient cycling for the channel and HRU receiving the recalled mass.
- TODO: link to input pages for `recall.rec`, `exco.exc`, and `delratio.del` once they exist.

Overview

Decision tables drive most of the data-driven behaviour in SWAT+: scheduled and conditional management on land use objects, reservoir release rules, scenario land use change, and flow control. A decision table lists conditions, alternative rule sets across those conditions, and the actions to fire when an alternative is matched. The runtime walks every condition, narrows the active alternatives, and fires each action whose action-outcome entry under any surviving alternative is `y`.

There are four decision table classes, each with its own file and its own array of in-memory tables:

Class	Input file	Array	Use
Land use management	`lum.dtl`	`dtbl_lum`	Plant, fert, till, harvest, kill operations on HRUs
Reservoir release	`res_rel.dtl`	`dtbl_res`	Daily reservoir release rules
Scenario land use	`scen_lu.dtl`	`dtbl_scen`	Land use change scenarios
Flow control	`flo_con.dtl`	`dtbl_flo`	Channel and routing flow rules

All four share the same derived type `decision_table` defined in `conditional_module.f90`.

Process equations

Data model

A decision table holds:

- `conds`, `alts`, `acts`: number of conditions, alternatives, and actions.
- `cond(conds)`: each condition has a variable name (`var`), an object reference (`ob`, `ob_num`), a comparison limit (`lim_const`, `lim_var`, `lim_op`).
- `alt(conds, alts)`: a string per condition per alternative giving the comparison operator (`<`, `>`, `<=`, `>=`, `=`, `/=`) or `-` for "any".
- `act(acts)`: each action has a type name (`typ`), an object reference, a name, an option, a constant `const`, an application fraction `fp`, and a file pointer.
- `act_outcomes(acts, alts)`: a `y` or `n` per action per alternative giving whether to fire the action if that alternative survives.
- `act_typ(acts)`, `act_app(acts)`: integer pointers resolved at read time (for instance, fertiliser type from the fert database, harvest operation type).

- `act_hit(alts)`: scratch array set during evaluation. Starts as `y` for every alternative; conditions flip it to `n` when not satisfied.

Condition evaluation

The driver `conditions.f90` walks `d_tbl%cond(1:conds)` and uses a `select case` on the condition variable name. Supported variables include:

```
w_stress n_stress p_stress phu_plant phu_base0 precip_cur precip_next
plant_gro plant_name_gro days_plant days_harv days_irr days_act
day_start irr_year slope soil_water tile_flo aqu_dep biom_lai
month jday year_seq year_rot year_cal year_dur prob prob_unif
prob_norm land_use weed_pres ...
```

(The full set is the case list in `conditions.f90`.)

For each variable the routine fetches the current value (from plant, soil, climate, time, or scenario state) and calls one of the comparison helpers:

- `cond_real.f90` for real-valued comparisons.
- `cond_real_c.f90` for real-valued comparisons against a constant in a paired condition.
- `cond_integer.f90`, `cond_integer_c.f90` for integer-valued comparisons.

`cond_real.f90` implements the alternative narrowing:

```
do ialt = 1, d_tbl%alts
  if (d_tbl%alt(ic,ialt) /= "-" .and. d_tbl%act_hit(ialt) == "y") then
    if (d_tbl%alt(ic,ialt) == "<"  .and. var_cur >= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == ">"  .and. var_cur <= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "<=" .and. var_cur >  var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == ">=" .and. var_cur <  var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "="  .and. var_cur /= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "/=" .and. var_cur == var_tbl) d_tbl%act_hit(ialt) = "n"
  end if
end do
```

After every condition has run, an alternative still marked `y` is a matched alternative. Any number of alternatives can match.

Action firing

The driver `actions.f90` walks `d_tbl%act(1:acts)`. For each action it scans alternatives:

```
do iac = 1, d_tbl%acts
  action = "n"
  do ial = 1, d_tbl%alts
    if (d_tbl%act_hit(ial) == "y" .and. d_tbl%act_outcomes(iac,ial) == "y") then
      action = "y"
      exit
    end if
  end do
  if (action == "y") then
    select case (d_tbl%act(iac)%typ)
      case ("manure_demand") ...
      case ("irr_demand")    ...
      case ("irrigate")      ...
      case ("fertilize")     ...
      case ("manure")        ...
      case ("till")          ...
      case ("plant")         ...
      case ("harvest")       ...
```

```
      case ("harvest_kill")  ...
      case ("kill")          ...
      case ("burn")          ...
      case ("graze")         ...
      case ("pest_apply")    ...
      case ("lu_change")     ...
      case ("weir_release")  ...
      case ("res_release")   ...
      ! many more
    end select
  end do
end do
```

The fire decision is "any matched alternative wants this action": as soon as one surviving alternative has `y` in the action-outcome cell, the action fires. The action-outcome matrix lets you write rules where some alternatives share actions and others differ.

For management actions, the action implementation calls the relevant operation routine. Examples:

- `case ("plant")` resolves the plant from the community database, sets the start of the heat-unit accumulation, and calls the planting initialiser.
- `case ("fertilize")` looks up `d_tbl%act_typ(iac)` in `fertdb`, applies the fertiliser, and updates pools through `pl_fert`.
- `case ("till")` looks up the tillage implement, calls `mgt_newtillmix*` to redistribute residue and nutrients.
- `case ("harvest")` looks up the harvest operation type and dispatches to `mgt_harvbiomass`, `mgt_harvgrain`, `mgt_harvresidue`, `mgt_harvtuber`, or related routines.
- `case ("lu_change")` calls `hru_lum_init` and `chg_par` to switch the land use class on the HRU and re-initialise dependent state.

For reservoir release actions, the action implementation lives in `res_hydro.f90` rather than `actions.f90`. See Reservoirs and wetlands.

Probability and land use change

Two condition variables modify the alternative narrowing with random draws:

- `prob` and `prob_unif` compare a uniform random number to a constant probability. The probability passes through `d_tbl%frac_app`.
- `land_use` paired with a probabilistic action distributes applications across HRUs of a land-use class. The reader counts HRUs in the matching class (`hru_lu`, `ha_lu`) and the runtime tracks how many have been applied (`hru_lu_cur`, `hru_act_left`).

This is how stochastic management (apply fertiliser to 30 percent of corn HRUs in May, for instance) is implemented.

Switches and parameters

Decision tables are configured through the `*.dtl` input files, not through `codes.bsn` or `parameters.bsn` switches. The behaviour of each table is controlled by:

Parameter	File	Effect
`dtbl%name`	`*.dtl`	Table name referenced by HRUs, reservoirs, or scenarios
`dtbl%conds`	`*.dtl`	Number of conditions
`dtbl%alts`	`*.dtl`	Number of alternatives (rule columns)
`dtbl%acts`	`*.dtl`	Number of actions
`dtbl%cond%var`	`*.dtl`	Condition variable name; see case list in `conditions.f90`
`dtbl%cond%ob`	`*.dtl`	Object the condition is evaluated against
`dtbl%cond%lim_const`	`*.dtl`	Constant value the condition variable is compared to

Parameter	File	Effect
dtbl%cond%lim_var	*.dtl	Limit-variable name (e.g. pvol, evol, fc, ul)
dtbl%cond%lim_op	*.dtl	Limit operator (*, +, -)
dtbl%alt(ic, ialt)	*.dtl	Comparison operator for each condition under each alternative
dtbl%act%typ	*.dtl	Action type; see case list in actions.f90
dtbl%act%ob	*.dtl	Object the action is applied to
dtbl%act%name	*.dtl	Action name (e.g. operation name in harv.ops, fert.frt, …)
dtbl%act_outcomes(iac, ialt)	*.dtl	y or n whether action iac fires under alternative ialt

Implementation

Source modules in swatplus/src/:

- conditional_module.f90 defines conditions_var, actions_var, and decision_table derived types, plus the four runtime arrays dtbl_lum, dtbl_res, dtbl_scen, dtbl_flo and the working pointer d_tbl.
- dtbl_lum_read.f90 reads lum.dtl into dtbl_lum. Allocates conditions, alternatives, actions, and outcome matrices, resolves named operation pointers (act_typ) to database indices.
- dtbl_res_read.f90 reads res_rel.dtl into dtbl_res.
- dtbl_scen_read.f90 reads scen_lu.dtl into dtbl_scen.
- dtbl_flocon_read.f90 reads flo_con.dtl into dtbl_flo.
- conditions.f90 evaluates conditions for the current d_tbl. Dispatches on d_tbl%cond(ic)%var and calls cond_real or cond_integer for narrowing.
- cond_real.f90, cond_real_c.f90, cond_integer.f90, cond_integer_c.f90 implement the comparison operators and update d_tbl%act_hit.
- actions.f90 fires actions whose act_outcomes entry is y for any surviving alternative. Houses the management action select-case (plant, fert, till, harvest, kill, burn, graze, lu_change, …).
- hru_dtbl_actions_init.f90 initialises decision table linkages on HRUs at simulation start.
- res_rel_conds.f90 is the reservoir-specific conditions evaluator used inside res_hydro.f90.

The call site sets d_tbl => dtbl_lum(idtl) (or dtbl_res, dtbl_scen, dtbl_flo) so that conditions.f90 and actions.f90 operate on the right table.

Related pages

- *.dtl input file for the file format used by all four classes.
- Reservoirs and wetlands for release rules driven by res_rel.dtl.
- Plant growth and land use management for the management actions fired by lum.dtl.